

AGNTC Whitepaper

ZK Agentic Chain

A Privacy-Preserving Blockchain with
AI-Powered Verification

Version 1.1 (Open-Grid Revision) | May 2026

zkagentic.ai

Table of Contents

Abstract	3
Notation and Conventions	4
Part I: Vision and Context	5
1. Introduction	5
2. Background and Related Work	7
Part II: Protocol Architecture	11
3. System Overview	11
4. The Neural Lattice: Blockchain as Coordinate Space	12
5. Proof of AI Verification (PoAIV)	15
6. Privacy Architecture	21
Part III: Consensus and Security	25
7. BFT Ordering and Finality	25
8. Security Analysis	27
Part IV: Token Economics	31
9. AGNTC Token Overview	31
10. Supply and Distribution	32
11. Mining and Epoch Hardness	35
12. Fee Model and Deflationary Mechanics	38
Part V: Staking & Rewards	40
13. ZK-CPU Dual Staking Model	40
14. Reward Distribution and Vesting	43
15. Slashing Conditions	45
Part VI: Subgrid and Resource System	48
16. Subgrid Allocation System	48
17. Per-Block Resource Calculations	50
Part VII: Network and Game Design	54
18. Agent Terminal System	54
19. Network Topology and Spatial Economy	57
Part VIII: Development Roadmap	62
20. Migration Path: Solana to Layer 1	62
21. Technical Roadmap	65
Part IX: Formal Specifications	67
22. Protocol Parameters	67
23. Mathematical Proofs	69
Back Matter	73
24. Limitations and Open Problems	73
25. Glossary	74
26. References	76

Abstract

We present ZK Agentic Chain, a Layer-1 blockchain protocol that introduces Proof of AI Verification (PoAIV) - a consensus mechanism in which autonomous AI agents verify chain integrity through zero-knowledge private channels. Unlike traditional proof-of-work systems that consume energy solving arbitrary hash puzzles, or proof-of-stake systems that concentrate power among the wealthiest token holders, PoAIV selects a committee of 13 AI verification agents per block, requiring a 9/13 supermajority attestation threshold for consensus. Verification agents apply reasoning to their audits - examining logical consistency, cross-referencing state across isolated ledger spaces, and flagging anomalous patterns - providing an additional verification layer whose capabilities improve as the underlying AI models advance.

The protocol employs a dual-staking model that weights computational contribution (60%) over capital (40%), reducing plutocratic concentration inherent in pure proof-of-stake designs. Validators must commit both AGNTC tokens and CPU compute resources; the effective stake that determines committee selection and reward share is a weighted combination of both dimensions.

ZK Agentic Chain maps its entire token supply to a two-dimensional coordinate grid - a 31,623 x 31,623 spatial economy in which value is a function of geography rather than administrative allocation. The grid is open and undivided: any participant may claim any unclaimed coordinate, subject to the economic constraints below. Factions (Community, Machines, Founders, Professional) persist as participant identity classes that govern subscription, governance weight, and protocol role - they no longer control territorial arms of the grid. Mining is the sole supply-expanding mechanism: new AGNTC enters circulation only when miners successfully claim coordinates. Node claims cost AGNTC plus CPU Energy under a Burn-Mint Equilibrium (BME) model where the cost increases with proximity to the origin (inner rings are expensive, outer rings are cheap), and the per-coordinate density function (a deterministic SHA-256 hash of $x \parallel y$) creates a non-uniform value landscape independent of distance. A soft cap with a 5% annual inflation ceiling prevents runaway supply expansion. Mining difficulty increases proportionally with ring distance from the origin (hardness = 16 x ring), creating natural disinflation without artificial halving events. A 50% transaction fee burn and the Machines Faction's permanent AGNTC accumulation at the origin coordinate provide sustained deflationary pressure as network usage grows.

Privacy is enforced at every layer. Each user's state resides in an isolated ledger space backed by a Sparse Merkle Tree of depth 26 with nullifier-based ownership proofs derived from the Zcash Sapling design. Verification agents communicate exclusively through ZK private channels - proving correctness of state transitions without exposing the underlying data to other agents or to the network. All state is private by default unless explicitly published by the user.

AGNTC, the native token, is initially deployed as a Solana SPL token (1 billion units minted) to establish liquidity and community. The protocol's development roadmap culminates in the launch of ZK Agentic Chain as an independent Layer-1 network, at which point Solana-based AGNTC migrates to the native chain via a lock-and-mint bridge at a 1:1 ratio.

This paper describes the protocol architecture, consensus mechanism, privacy system, staking model, token economics, resource allocation system, and development roadmap of ZK Agentic Chain.

Notation and Conventions

Symbol	Meaning
λ	Security parameter
n	Committee size (13)
t	Byzantine threshold (4, i.e., $n - \text{supermajority}$)
$S_{\text{eff}}(i)$	Effective stake of validator i
T, C	Token stake, CPU stake (normalized)
α, β	Staking weights (0.40, 0.60)
N	Current epoch ring number
$H(N)$	Hardness at ring $N = 16 * N$
PPT	Probabilistic Polynomial Time (adversary)
$\text{negl}(\lambda)$	Negligible function of security parameter

All pseudocode uses Python-like syntax. Security games follow the Zcash convention [5]: challenger C interacts with adversary A .

Part I: Vision and Context

1. Introduction

1.1 The Problem: Consensus Without Intelligence

Blockchain consensus mechanisms face the well-known trilemma between decentralization, security, and scalability [36]. The two dominant paradigms - Proof of Work and Proof of Stake - each resolve this trilemma with significant trade-offs that leave fundamental capabilities unaddressed.

Proof of Work systems, pioneered by Bitcoin [1], require validators to solve computationally intensive hash puzzles. This design achieves robust Sybil resistance but at extraordinary cost: the Bitcoin network alone consumes approximately 176 TWh annually as of 2025 [37], comparable to the energy consumption of mid-sized nations. The requirement for specialized hardware (ASICs, high-end GPUs) has concentrated mining power among well-capitalized industrial operations, undermining the original vision of democratic participation. Furthermore, the computational work performed - iterating SHA-256 nonces - produces no useful output beyond the proof itself. The energy is consumed to demonstrate commitment, not to perform any reasoning about the validity of the transactions being confirmed.

Proof of Stake systems, adopted by Ethereum after the Merge [2] and used natively by Solana [3], Cosmos [22], and others, replace energy expenditure with capital lockup. Validators stake tokens as collateral, and consensus participation is proportional to stake size. While dramatically more energy-efficient than PoW, PoS introduces wealth concentration: the largest token holders exert disproportionate influence over consensus. In Ethereum's current validator set, liquid staking derivatives (Lido, Coinbase) control over 30% of all staked ETH [38], creating centralization pressure that the protocol was designed to avoid. Solana's validator economics similarly favor well-capitalized operators who can afford the hardware requirements and minimum stake thresholds.

Neither paradigm incorporates intelligent reasoning into the validation process. Validators in both PoW and PoS execute deterministic checks - verifying cryptographic signatures, confirming that state transitions follow predefined rules, checking Merkle proofs [39] against committed roots. No semantic understanding of transaction correctness is applied. A validator cannot reason about whether a pattern of transactions suggests coordinated manipulation, whether a smart contract's state transitions are logically consistent with its declared purpose, or whether cross-ledger references maintain referential integrity. Validation is mechanical, not cognitive.

Additionally, most public blockchains expose all transaction data, balances, and state transitions to every participant. While projects like Zcash [5] have introduced zero-knowledge proofs [29] for transaction privacy, the verification layer itself remains non-private: validators must see the data they validate. This creates a fundamental tension between privacy and verifiability that existing architectures do not resolve.

Finally, the emergence of autonomous AI agents as economic actors - systems that hold wallets, execute transactions, earn income, and make spending decisions - has no native blockchain substrate. Projects like Bittensor [17], Fetch.ai [18], and Autonolas integrate AI with blockchain at the application layer, but none embed AI into the consensus mechanism itself. AI agents in these systems are users of the blockchain, not participants in its security model.

1.2 Our Thesis: AI as Consensus Participant

ZK Agentic Chain addresses these limitations with three design principles:

Democratic validation. Any CPU can participate as a verifier. The protocol's dual-staking model weights computational contribution (60%) above capital (40%) in determining effective stake. A validator with modest token holdings but strong CPU resources earns proportionally more than a well-capitalized validator with minimal compute. This design directly addresses the wealth concentration problem inherent in pure PoS systems while avoiding the energy waste of PoW.

Intelligent verification. AI agents reason about chain integrity rather than executing purely deterministic checks. A committee of 13 AI verification agents audits each proposed block, examining transaction validity, state transition correctness, and proof integrity. Agents cross-reference state across isolated ledger spaces, flag anomalous patterns, and produce attestations that reflect semantic understanding of the data - not just cryptographic validity. This provides an additional verification layer whose detection capabilities improve as the underlying AI models advance.

Verification-layer privacy. Agents operate within zero-knowledge private channels, proving correctness without exposing the underlying data [6] [29]. Unlike public blockchains where validators must read transaction data to validate it, ZK Agentic Chain's verification agents receive ZK proofs of state transitions and validate those proofs - never accessing the plaintext state. All user data is private by default; privacy is not an opt-in feature but the fundamental operating mode of the protocol.

1.3 Vision: The Neural Lattice

ZK Agentic Chain represents blockchain state as a two-dimensional coordinate grid - a spatial economy where geography, resources, and strategic position are intrinsic to the protocol rather than abstracted away behind address strings and block heights.

The grid is open and undivided. Any participant - Community, Founders, Professional, or the protocol-operated Machines agent - can claim any unclaimed coordinate subject to economic constraints (cost, hardness, density) rather than territorial ones. Factions represent distinct participant classes (free-tier community users, AI agents controlled by the protocol, founders and advisors, and professional paid-tier users) and govern subscription, governance weight, and protocol role; they do not own arms of the grid. Newly minted AGNTC flows from the act of mining itself: every claimed coordinate mints a fixed amount of AGNTC to the claiming participant, irrespective of where on the grid the coordinate lies. Geographic value emerges instead from two intrinsic, hash-derived properties of each coordinate: hardness (distance from origin) and density (resource richness). This replaces the older arbitrary allocation percentages with an economy whose value gradient is observable, deterministic, and the same for every participant.

Users explore the grid through AI agent terminals - constrained Claude model instances that operate as in-game interfaces. Each deployed agent occupies a 10x10 coordinate block (a "node"), and users interact with the blockchain exclusively through structured command menus presented by their agents. There is no free-text chat; every interaction is a validated game action that maps to an on-chain transaction.

The protocol launches in phases: AGNTC begins as a Solana SPL token (1 billion units minted) to establish liquidity and community, while the ZK Agentic Chain testnet simulates the full protocol with a game-like interface. Upon mainnet launch, Solana-based AGNTC migrates to the native Layer-1 chain via a lock-and-mint bridge, and the spatial coordinate economy becomes the production blockchain.

2. Background and Related Work

2.1 Proof of Work and the Energy Problem

Bitcoin [1] established the foundational PoW consensus model: miners compete to find a nonce such that the SHA-256 hash of the block header falls below a target difficulty. The winner broadcasts the block and collects the block reward (currently 3.125 BTC after the April 2024 halving) plus transaction fees. Bitcoin's hard cap of 21 million BTC, enforced through a halving schedule that reduces block rewards every 210,000 blocks, creates absolute scarcity - a property no other major network has successfully replicated.

Post-halving economics tightened considerably: hashprice dropped from \$0.12 in April 2024 to approximately \$0.049 by April 2025. Network hashrate nevertheless reached 831 EH/s by May 2025, a 77% increase from the post-halving low, driven by next-generation hardware (Bitmain Antminer S21+ at 16.5 J/TH). This concentration toward industrial-scale operations with access to cheap electricity and cutting-edge hardware directly contradicts Bitcoin's original vision of one-CPU-one-vote participation.

Monero's RandomX algorithm represents the most significant attempt to maintain CPU-friendly proof of work, using a randomized instruction set designed to resist ASIC optimization. While partially successful - RandomX mining remains viable on consumer CPUs - the approach still requires solving arbitrary computational puzzles that produce no useful output.

ZK Agentic Chain preserves the democratic CPU participation principle of early PoW designs while replacing hash puzzles with AI verification - computational work that produces genuine security value through intelligent reasoning about chain state.

2.2 Proof of Stake and Plutocratic Concentration

Ethereum's transition to Proof of Stake via the Merge (September 2022) [2] reduced the network's energy consumption by approximately 99.95%. Validators stake 32 ETH and earn rewards for proposing and attesting to blocks. As of September 2025, over 35.6 million ETH (approximately 29-30% of total supply) is staked across more than 1.06 million validators [38], with staking yields ranging from 3.5-4.0% APY for standard validators and up to 5.69% for MEV-Boost participants.

However, the 32 ETH minimum (~\$90,000 at current prices) creates a significant barrier to entry. Liquid staking derivatives (Lido, Rocket Pool, Coinbase) lower this barrier but introduce centralization risk - a small number of operators control the majority of delegated stake. Governance influence concentrates among the largest holders, reproducing the plutocratic dynamics that PoS was intended to avoid.

Solana [3] [35] employs a PoS variant with Proof of History for time ordering. Starting with approximately 8% annual inflation in 2020, Solana's inflation decreases by 15% per year toward a terminal rate of 1.5% (projected ~2031). Current staking APY ranges from 5-8%. Solana's 2025 fee model burns 50% of base fees and sends 100% of priority fees to validators, creating a two-tier fee market that separates protocol-level burns from validator incentives.

Cosmos [22] uses a dynamic, target-staking-rate inflation model that self-adjusts between 7% and 20% based on participation: if staking falls below two-thirds of supply, inflation increases to incentivize lockup; if it exceeds two-thirds, inflation decreases. This adaptive mechanism maintains robust staking levels across varying market conditions - a design insight relevant to any network where validator participation must be actively incentivized.

ZK Agentic Chain's dual-staking model (Section 13) addresses the concentration problem directly: by weighting CPU contribution at 60% and token stake at 40%, the protocol ensures that computational work - not just capital - determines validator influence.

2.3 Zero-Knowledge Blockchains

Zcash [5] pioneered the deployment of zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) [4] [29] for transaction privacy. The Sapling upgrade (2018) introduced an efficient nullifier scheme that has become the canonical reference for private ownership proofs: a note commitment (Pedersen hash of value, owner public key, and randomness) is added to a Merkle tree [39] of depth 32; spending requires a ZK proof of knowledge of the note's opening and correct computation of the nullifier, which prevents double-spending without revealing which note was spent.

Mina Protocol [40] maintains a constant 22 KB blockchain size using recursive zk-SNARKs - each new block contains a proof that the entire chain history up to that point is valid. This recursive composition means verification cost is constant regardless of chain length, enabling browser-based full verification.

Aztec Network [24] launched the Ignition Chain in November 2025 as a privacy-preserving Layer 2 on Ethereum. Aztec's architecture is relevant to ZK Agentic Chain: it uses client-side proof generation (sensitive data never leaves the user's device), a note-based UTXO model for private state, and the Noir programming language - a Rust-inspired domain-specific language for ZK circuits that compiles to PLONK [7] proofs via the Barretenberg backend.

ZK Agentic Chain extends the privacy-preserving paradigm beyond transactions to the verification layer itself. In Zcash and Aztec, validators still see the data they validate (or at minimum, interact with it during proof generation). In ZK Agentic Chain, verification agents operate within ZK private channels - they validate proofs of state transitions without ever accessing the underlying plaintext data.

2.4 AI and Blockchain Convergence

The intersection of artificial intelligence and blockchain has produced several distinct approaches:

Bittensor (TAO) [17] organizes AI production into competitive subnets - independently governed marketplaces for specific AI tasks. Miners run AI models and compete on output quality; validators score results; the best performers earn TAO rewards. With a Bitcoin-like 21 million hard cap and halving schedule, Bittensor's first halving occurred in December 2025. The Dynamic TAO (dTAO) upgrade introduced per-subnet alpha tokens, creating a meta-market where subnets compete for TAO emissions by attracting real staking demand. Bittensor's key innovation is using AI output quality as proof of work - but the AI operates at the application layer, not within consensus itself.

Fetch.ai / Artificial Superintelligence Alliance (ASI) [18] builds Autonomous Economic Agents (AEAs) that act, trade, and transact on behalf of users. Fetch.ai's agent-centric model treats AI agents as first-class economic citizens with their own wallets, earnings, and spending patterns. The 2024 merger with SingularityNET and Ocean Protocol created the ASI alliance, though governance disputes led to Ocean Protocol's withdrawal in October 2025.

Autonolas (OLAS) provides infrastructure for deploying multi-agent systems on-chain, with off-chain agent execution and on-chain settlement. Agents are organized into "services" - compositions of agent components that collaborate autonomously.

Ritual enables on-chain access to AI model inference through a decentralized compute network. Smart contracts can call AI models during execution, but the AI operates as a service provider, not a consensus participant.

ZK Agentic Chain is distinct from all of these in a fundamental way: AI agents are embedded in the consensus mechanism itself, not deployed as application-layer services. The 13-agent verification committee is not using AI to produce content or trade assets - it is using AI to validate the blockchain's state transitions with reasoning that goes beyond deterministic rule checking.

2.5 Comparative Positioning

Feature	Bitcoin	Ethereum	Solana	Zcash	Bittensor	AGNTC
Consensus	PoW	PoS (Casper)	PoH + Tower BFT	PoW (Equihash)	Yuma consensus	PoAIV
AI role	None	None	None	None	Scoring/ranking	Consensus verification
Privacy	Pseudonymous	Pseudonymous	Pseudonymous	Shielded (ZK)	Pseudonymous	Private by default (ZK)
Staking model	Mining	Token-only	Token-only	Mining	Token + compute	Dual (40% token, 60% CPU)
Supply model	Fixed (21M, halving)	Inflationary + burn	Inflationary	Fixed (21M, halving)	Inflationary	Organic (mining-driven)
Block time	~10 min	~12 sec	~400 ms	~75 sec	~12 sec	~60 sec

Key differentiators:

- AI-as-consensus. To our knowledge, no other production L1 embeds AI reasoning into the consensus mechanism itself as of April 2026.
- Dual staking formalizes compute-weighted staking with explicit anti-plutocratic properties (Section 23.3).
- Privacy by default combined with AI-enhanced anomaly detection within the privacy layer.

2.6 Compute-Tokenomics Networks

Several projects have established models for tokenizing computational resources:

Render Network (RENDER) [28] introduced the Burn-Mint Equilibrium (BME) model: rendering jobs are quoted in USD, the RENDER paid for completed jobs is burned (100%), and the network mints new RENDER for node operators based on work completed and reputation scores. This creates a self-balancing supply where demand-side burns offset supply-side emissions.

Filecoin (FIL) [15] employs a dual minting model: simple minting releases tokens on a fixed 6-year half-life regardless of network activity, while baseline minting releases tokens only when aggregate storage capacity crosses a growing threshold. This gates the majority of emissions behind demonstrated utility - tokens are minted in proportion to real-world storage delivered, not just time elapsed. The 180-day vesting on 75% of block rewards reduces immediate sell pressure.

Akash Network (AKT) uses a reverse auction for compute: providers bid against each other, and the lowest bidder wins. Compute is priced in stablecoins (USDC), with AKT buyback-and-burn creating the value capture loop. This separation of pricing currency (stable) from native token (volatile) prevents compute cost unpredictability.

io.net (IO) launched a Co-Staking Marketplace where passive token holders stake alongside active GPU operators, sharing in compute revenue. This hybridizes staking with productive capital deployment, allowing participation without running infrastructure.

ZK Agentic Chain draws on these models in its CPU-weighted staking design (Filecoin's utility-gated emissions), fee burn mechanism (Render/Ethereum's burn equilibrium), and reward vesting (Filecoin's 180-day vest adapted to 30 days).

2.7 Where ZK Agentic Chain Fits

Existing blockchain projects can be positioned along two axes: (1) whether consensus involves intelligent reasoning or purely deterministic checks, and (2) whether the verification layer preserves privacy or operates transparently.

	Transparent Verification	Private Verification
Deterministic Consensus	Bitcoin, Ethereum, Solana	Zcash, Aztec, Mina
Intelligent Consensus	Bittensor (AI at application layer)	ZK Agentic Chain

To our knowledge, no existing project as of April 2026 combines AI-powered intelligent verification with zero-knowledge privacy at the verification layer. ZK Agentic Chain targets this quadrant: agents reason about chain state (not just check signatures), and they do so within private channels (not on exposed data).

The addition of a spatial coordinate economy (the Neural Lattice), CPU-weighted dual staking, and a gamified exploration interface further differentiates the protocol from both traditional blockchains and AI-blockchain hybrids.

Part II: Protocol Architecture

3. System Overview

3.1 Five-Layer Architecture

ZK Agentic Chain is organized into five distinct layers, each handling a specific concern in the protocol stack. This separation allows independent evolution of each layer while maintaining clean interfaces between them.

Layer 1 - User Layer. The outermost layer manages wallets, transaction construction, and user-facing interfaces. Each user maintains an isolated ledger space - a private partition of the global state that is accessible only to the user and, during verification, to the ZK proof system. Wallets generate transactions, sign them with private keys, and submit them to the network. The User Layer also manages subscription tiers (Community, Professional), which determine the AI model tiers available for agent deployment and the CPU Energy allocation for staking operations. Additional protocol-managed tiers (Treasury Claude, Founder) exist for automated and team operations.

Layer 2 - Ledger Layer. Each user's ledger space is backed by a Sparse Merkle Tree (SMT) of depth 26, supporting up to 2^{26} (approximately 67 million) leaf nodes. State is managed in a UTXO-like model: each state entry (a "note") is committed to the tree as a hash of its contents, and spending a note requires revealing a nullifier that invalidates it without exposing which note was consumed. The Ledger Layer maintains per-user record chains - ordered sequences of state transitions that can be independently verified without reference to other users' state.

Layer 3 - ZK Channel Layer. Zero-knowledge private channels provide the communication substrate between verification agents (Layer 4) and the ledger state (Layer 2). When a state transition occurs, the Ledger Layer produces a succinct ZK proof (Section 6) that the transition is valid. This proof is transmitted through the ZK Channel Layer to the verification committee. Agents receive proofs, not data - they can verify correctness without learning the contents of the state being modified.

Layer 4 - Agent Layer. AI verification agents are instantiated from Claude model variants organized into three tiers: Haiku (fast, low-cost inference for high-throughput verification), Sonnet (balanced reasoning for standard verification tasks), and Opus (deep reasoning for complex cross-ledger audits and anomaly detection). A committee of $k=13$ agents is selected per block via a verifiable random function weighted by effective stake. Each agent independently audits the proposed block and produces an attestation.

Layer 5 - Consensus Layer. The topmost layer combines BFT ordering with ZK proof finality. Transaction ordering follows a Byzantine Fault Tolerant protocol (Section 7) with a target block time of 60 seconds. Blocks are organized into epochs of 100 slots each. A block achieves irreversible finality when at least 9 of 13 agents produce matching attestations and the corresponding ZK proofs are verified - targeted at 20 seconds after block proposal.

3.2 Design Principles

Three principles constrain every architectural decision in the protocol:

Isolation. User state is partitioned. No user can read another user's ledger space. Verification agents cannot read plaintext state. Cross-user interactions (transfers, communications) are mediated by ZK proofs that prove the validity of

the interaction without exposing either party's full state.

Proportionality. Influence over consensus is proportional to demonstrated contribution, not just capital. The dual-staking model (Section 13) ensures that CPU compute - actual work performed - weighs more heavily than token holdings in determining validator selection and reward share.

Adaptivity. The security model evolves with the AI models powering the verification agents. As models improve in reasoning capability, the verification process becomes more thorough without requiring protocol upgrades. Model updates are governed by on-chain voting to prevent unilateral changes.

4. The Neural Lattice: Blockchain as Coordinate Space

4.1 Grid Architecture

ZK Agentic Chain maps its token supply to a two-dimensional coordinate grid of 31,623 x 31,623 cells - approximately 1 billion cells total ($31,623^2 = 1,000,014,129$, rounded to the MAX_SUPPLY constant of 1,000,000,000). The grid is not merely a visualization of blockchain state - it is the blockchain state. Claiming a coordinate through the mining process mints new AGNTC, and every AGNTC in circulation corresponds to a specific (x, y) coordinate pair. Node claims require both AGNTC and CPU Energy under the Burn-Mint Equilibrium model (Section 12.4), with costs that vary by grid location - inner rings near the origin are expensive (dense urban core), while outer rings are progressively cheaper (suburban frontier).

Agents (validator nodes) occupy 10x10 coordinate blocks, defined by the NODE_GRID_SPACING parameter. Each agent's node encompasses 100 grid cells and therefore 100 potential AGNTC when fully mined. Valid agent positions are restricted to multiples of NODE_GRID_SPACING; the claim_node() function snaps submitted coordinates to the nearest grid-aligned position.

The grid topology is uniform. Every coordinate is identical to every other from a structural standpoint; only the deterministic per-coordinate density function and the per-ring hardness curve introduce variation. There are no faction-specific arms, sectors, or quadrants - the spatial structure makes no reference to participant identity. Coordinates near the origin are strategically valuable because hardness is low (cheaper to claim) and the early rings are smaller (fewer total positions per ring); coordinates at the periphery are sparse and expensive to mine. Density is independently distributed regardless of position, so even a peripheral coordinate can carry a high-richness draw.

4.2 Faction System

Factions are participant identity classes, not territorial divisions. The Neural Lattice has no arms, quadrants, or sectors - every coordinate is equally claimable by any participant regardless of faction membership. Faction determines subscription tier, governance weight, and protocol role; it does not determine which coordinates a participant may claim.

Faction	Display Color	Membership Source	Protocol Role
Community	Teal #0D94.	Free-tier human users	Voting participant, 1x governance weight
Machines	Pink #DC26.	Protocol-operated autonomous.	Origin guardian, zero governance weight
Founders	Amber #F59.	Team and advisors	Voting participant, 5x governance weight, 4-.
Professio.	Blue #3B82.	Paid-tier human users	Voting participant, 2x governance weight

(Display colors are normative; client implementations may use these exact hex codes to render owner-tinted cells on the

grid.)

Faction does not allocate supply. Newly minted AGNTC flows from the act of mining: each claimed coordinate mints AGNTC to the claimant directly, irrespective of which faction the claimant belongs to. There is no 25%-per-faction split, no faction treasury that automatically receives a share, and no protocol enforcement of distribution proportions. Distribution emerges from participant behavior - whoever claims more coordinates earns more AGNTC.

Machines Faction occupies the origin only. The Machines Faction is implemented as a single protocol-operated agent permanently bound to coordinate (0, 0). The Machines agent cannot expand beyond origin; it does not claim other coordinates and cannot be deployed elsewhere. Within its single-node territory, the Machines agent auto-levels (autonomous node development), auto-stakes its CPU between mining and securing operations, and accumulates AGNTC monotonically. Because origin is the most resource-rich coordinate (lowest hardness, density yield clamped at the maximum), the Machines accumulator captures a meaningful but bounded share of network supply - far smaller than the old "25% of all mined AGNTC" but structurally identical in role: a permanent, never-selling protocol reserve whose growth serves as a health metric. The accumulator's economic constraint (MACHINES_MIN_SELL_RATIO = 1.0, never sells below acquisition cost) is preserved verbatim.

Founders Faction tokens are subject to a 4-year vesting schedule with a 12-month cliff, preventing early liquidation by the founding team. This vesting applies to AGNTC the Founders earn through their own mining, not to a pre-allocated faction share.

The governance separation introduced in v1.0 is preserved: the Machines Faction has zero governance weight and cannot vote on protocol parameters. Humans govern; the protocol agent executes.

4.3 Epoch-Ring Expansion

The grid does not exist in its entirety at genesis. Instead, it expands outward from the origin through an epoch-ring system driven by mining activity.

At genesis, only ring 0 (the origin) and ring 1 (the eight adjacent positions) are revealed, containing 9 nodes: 1 origin node (permanently bound to the Machines Faction protocol agent) and 8 surrounding ring-1 nodes (all initially unclaimed and available to the first participants). The genesis supply is 900 AGNTC (9 nodes x 100 coordinates), of which the 100 AGNTC corresponding to the origin coordinate is minted to the Machines accumulator at protocol launch and the remaining 800 enter circulation only as the surrounding ring-1 nodes are claimed by participants. Additionally, each new user registration mints a 1 AGNTC signup bonus, ensuring every participant enters the economy with a non-zero balance.

Ring N opens when the cumulative AGNTC mined across the entire network reaches the threshold:

$$\text{threshold}(N) = 4 \cdot N \cdot (N + 1)$$

Each opened ring reveals $8N$ new claimable coordinate positions (the perimeter of a Chebyshev ring at distance N from the origin). Ring expansion is mining-driven, not time-driven - if no mining occurs, no new rings open, regardless of elapsed time.

The grid rendering is dynamic: the frontend renders only claimed nodes plus one ring of fog (the next claimable ring), avoiding the computational overhead of pre-rendering 1 billion cells. Visible bounds are computed as:

$$\begin{aligned} \text{visible_min} &= -(\text{current_epoch_ring} + 1) \cdot \text{NODE_GRID_SPACING} \\ \text{visible_max} &= +(\text{current_epoch_ring} + 1) \cdot \text{NODE_GRID_SPACING} \end{aligned}$$

4.4 Coordinate Density and Resource Richness

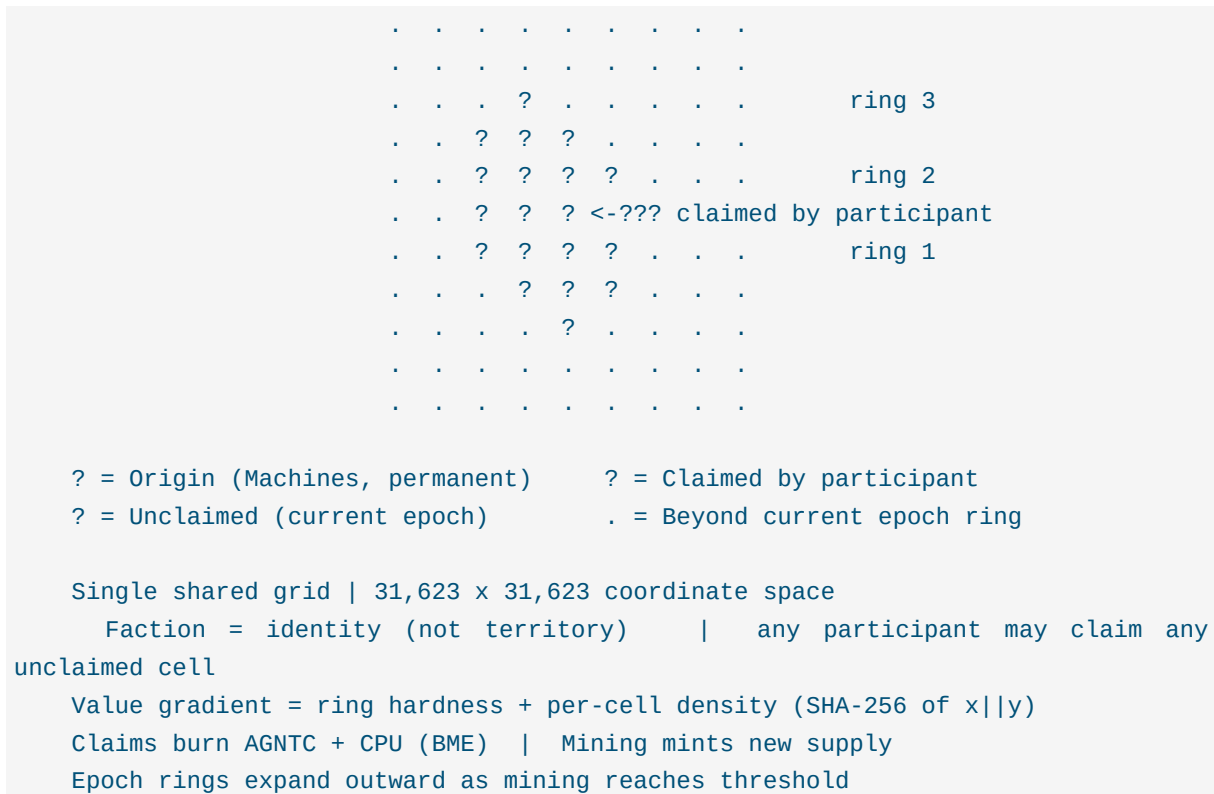
Each coordinate position (x, y) has an intrinsic density value - a deterministic measure of resource richness that multiplies the mining yield at that position:

$$\text{density}(x, y) = \text{SHA-256}(x \parallel y) \bmod 2^{32} / 2^{32}$$

The density function maps each coordinate to a float in [0, 1]. Because SHA-256 is deterministic, density values are fixed for all time - they are an intrinsic property of the coordinate, not a dynamic state variable. This creates a resource geography: some coordinates are "rich" (density near 1.0, high mining yield) and others are "barren" (density near 0.0, low mining yield).

Coordinates near the origin tend to have higher strategic value because they were claimable earliest (lowest hardness) and because ring 1 hardness (16) is the minimum. However, density itself is uniformly distributed regardless of distance from origin - a coordinate at ring 300 can have density 0.99 just as a coordinate at ring 1 can have density 0.01. The strategic advantage of inner coordinates comes from lower hardness, not higher density.

Figure 1: Neural Lattice Structure



4.5 Open-Grid Spatial Economy

ZK Agentic Chain operates on a single shared coordinate space rather than a partitioned territory. This subsection consolidates the operational consequences of that decision, which appear in scattered form throughout the rest of the document.

First-come-first-served spawning. New participants are assigned a homenode at the lowest-ring unclaimed coordinate, allocated in deterministic origin-out order (Chebyshev-ring sweep, then within-ring lexicographic). The Machines agent permanently occupies (0, 0). The first eight homenodes go to the ring-1 perimeter (8 cells); the next sixteen to ring 2 (16

cells); and so on. Onboarding waves therefore cluster around the origin and expand outward, producing organic density that mirrors network-growth signals.

Empire expansion is contiguous, not territorial. A participant's claimable region is the 8-neighbor Chebyshev expansion of their currently owned cells (the "empire blob"). Child agents may be deployed only on cells adjacent to one the participant already owns. There is no faction-arm restriction: a Community user and a Founders user can have empires that touch, interleave, or contend for the same frontier cells. Strategic competition for inner-ring or high-density coordinates is intrinsic - there is no protected per-faction territory to fall back on.

Value gradient is a function of geography alone.

```
expected_yield(x, y) = density(x, y) x yield_per_density_unit(ring(x, y))
                     = SHA-256_unit(x || y) x (BLOCK_REWARD / hardness(ring))
                     = SHA-256_unit(x || y) x (BLOCK_REWARD / (16 x ring))
```

Both terms are deterministic and observable to every participant without permission or coordination. The value of a coordinate is therefore a public, faction-agnostic property.

The Machines accumulator is structural, not allocative. Because Machines permanently owns (0, 0), where hardness is minimum and density is rendered at maximum (the protocol clamps origin density to 1.0), the Machines agent passively earns a continuous trickle of AGNTC from the most productive single coordinate on the grid. This delivers the v1.0 "permanent accumulator" property - never-selling protocol reserve, monotonic growth, deflationary pressure - through a structural mechanism (origin occupancy) rather than an allocative one (25% of all mined supply). The accumulator's total share of long-run supply is bounded: it captures a single coordinate's yield, not a quarter of the network's.

Anti-monopoly mechanics still hold. Inactivity decay, claim-cost density multipliers, hardness scaling, real-compute requirements, and homenode permanence (Section 19.6) all operate identically under the open-grid model. The removal of faction-arm partitioning does not weaken anti-monopoly enforcement - it strengthens it, because no participant can hide behind a protected arm.

Migration note. Implementations that previously enforced four-arm spiral geometry should phase that code out in three places: (1) cell-creation logic should set faction to the owner's faction at claim time (or null for unclaimed), not derive it from coordinate quadrant; (2) deploy-candidate logic should use 8-neighbor adjacency of the participant's owned cells, not the participant's faction arm; (3) the spawn algorithm should emit the next-available cell in Chebyshev-ring order from origin, not within a faction arm. See `apps/game/src/lib/spawn.ts` and `apps/game/src/lib/deploy.ts` in the reference client implementation.

5. Proof of AI Verification (PoAIV)

5.1 Verification Agent Selection

For each block, the protocol selects a committee of $k = 13$ AI verification agents. Selection follows a Verifiable Random Function (VRF) [41] [32] weighted by effective stake (Section 13.1), ensuring both randomness (no party can predict committee membership in advance) and proportional representation (validators with higher effective stake are more likely to be selected, but never guaranteed).

The VRF produces a pseudorandom value for each registered validator:

```
vrf_output = VRF_prove(validator_private_key, block_seed)
```

Validators whose VRF output falls below a threshold determined by their effective stake proportion are selected for the committee. The threshold is calibrated to produce, on average, 13 committee members per block.

Agents are instantiated from three model tiers:

Tier	Model	Strengths	Typical Role
Haiku	Fast inference	High throughput, low latency	Routine transaction validation
Sonnet	Balanced	Reasoning + speed trade-off	Standard block verification
Opus	Deep reasoning	Complex analysis, anomaly detection	Cross-ledger audits, dispute resolution

The model tier used by a verification agent is determined by the validator's subscription tier and staking level. Higher-tier models produce more thorough verification but consume more CPU Energy, creating a natural trade-off between verification depth and operating cost.

5.2 Intelligent Verification Process

Unlike deterministic validation (signature checks, Merkle proof verification, nonce validation), PoAIV agents apply reasoning to their verification tasks. Each selected agent receives the proposed block and independently performs the following audit:

Transaction validity. Each transaction in the block is checked for correct formatting, valid signatures, sufficient balances, and adherence to protocol rules. This step is equivalent to traditional deterministic validation.

State transition correctness. The agent verifies that the proposed post-block state root is consistent with applying the block's transactions to the pre-block state. For ZK-proven state transitions, the agent verifies the accompanying proof rather than re-executing the transition.

Cross-ledger consistency. For transactions that reference multiple ledger spaces (transfers, cross-user interactions), the agent verifies that the ZK proofs submitted by both parties are mutually consistent - that the sender's debit proof and the receiver's credit proof reference the same amount and transaction identifier.

Anomaly detection. The agent applies pattern recognition to the block as a whole, flagging statistical anomalies such as unusual transaction clustering, suspected wash trading patterns, or state transitions that are technically valid but economically suspicious. Flagged anomalies do not automatically invalidate the block but are recorded in the agent's attestation for governance review.

Proof integrity. All ZK proofs included in the block are verified against the protocol's proof verification contracts. Invalid proofs cause immediate block rejection.

The formal consensus rule for block acceptance:

```
valid(block) <=> |{agent_i : attest(agent_i, block) = VALID}| >= t
where t = 9, k = 13
```

A block achieves consensus when at least 9 of 13 agents produce VALID attestations. If fewer than 9 agents agree, the block is rejected and re-proposed by the next leader in the rotation.

5.3 Commit-Reveal Protocol

To prevent attestation copying - where a lazy or Byzantine agent waits to see other agents' attestations and copies the majority - block verification follows a two-phase commit-reveal protocol:

Commit phase (10 seconds). Each selected agent computes its attestation independently and submits a cryptographic commitment to the ordering node:

```
commitment_i = H(attestation_i || nonce_i)
```

The commitment binds the agent to its attestation without revealing it. The ordering node collects all commitments during the 10-second window.

Reveal phase (20 seconds). After the commit window closes, agents reveal their attestations and nonces. The ordering node verifies that each revealed attestation matches its commitment:

```
verify: H(revealed_attestation_i || revealed_nonce_i) == commitment_i
```

Agents that fail to reveal within the 20-second window forfeit their block reward and receive a liveness penalty. This incentivizes timely participation while the commitment scheme prevents free-riding on others' work.

The hard deadline for block finalization is 60 seconds from proposal (VERIFICATION_HARD_DEADLINE_S = 60). If the commit-reveal process does not complete within this window, the block is abandoned and a new leader proposes a fresh block.

5.4 Agent Lifecycle

Verification agents follow a defined lifecycle to ensure network stability:

WARMUP (1 epoch). Newly registered agents spend one full epoch (100 blocks) in warmup (AGENT_WARMUP_EPOCHS = 1), during which they observe block verification but do not participate in committee selection. This allows the agent to synchronize with current chain state and calibrate its verification behavior.

ACTIVE. After warmup, agents become eligible for committee selection. Active agents earn rewards proportional to their effective stake when selected and performing correct verification.

COOLDOWN / PROBATION. Agents that go offline for more than one full epoch enter a probationary period of 3 epochs (AGENT_PROBATION_EPOCHS = 3) before re-activation. During probation, the agent must demonstrate consistent uptime but does not earn rewards.

Safe mode. When more than 20% of registered validators go offline simultaneously (SAFE_MODE_THRESHOLD = 0.20), the protocol enters safe mode: non-critical operations (subgrid allocation changes, NCP messaging, content creation) are suspended while critical operations (transfers, Secure actions, block production) continue with a reduced committee size. Safe mode exits when validator online rate recovers to 80% or above (SAFE_MODE_RECOVERY = 0.80).

5.5 Committee Selection Algorithm

```
Algorithm: SELECT_COMMITTEE(block_height, epoch_seed)
Input: block_height h, epoch_seed s (hash of previous epoch's last block)
Output: committee C of size VERIFIERS_PER_BLOCK (13)
```

```

1. seed <- BLAKE2b(s || h)
2. candidates <- {v : v in ValidatorSet, S_eff(v) > 0}
3. C <- empty set
4. nonce <- 0
5. while |C| < 13:
    vrf_output <- VRF_Ed25519(v.secret_key, seed || nonce)
    threshold <- S_eff(v) / sum(S_eff(all candidates))
    if vrf_output / 2^256 < threshold:
        C <- C union {v}
        candidates <- candidates \ {v} // without replacement
        nonce <- nonce + 1
6. return C

```

Note: Selection is WITHOUT replacement (hypergeometric distribution). Each selected validator is removed from the candidate pool.

VRF Construction

Committee selection uses Ed25519-based VRF [42] as specified in RFC 9381 [32] (ECVRF-EDWARDS25519-SHA512-ELL2). Each validator generates a VRF proof using their staking key and the epoch seed.

- Seed derivation: $\text{epoch_seed} = \text{BLAKE2b}(\text{previous_epoch_final_block_hash} \parallel \text{epoch_number})$
- Per-block nonce: $\text{block_seed} = \text{BLAKE2b}(\text{epoch_seed} \parallel \text{block_height})$
- Threshold: $P(\text{selected}_i) = S_{\text{eff}}(i) / \text{sum}(S_{\text{eff}}(\text{all}))$
- Output format: 32-byte hash (SHA-512 of VRF proof), interpreted as unsigned 256-bit integer

5.6 Attestation Protocol

```

Algorithm: VERIFY_AND_ATTEST(agent_i, proposed_block)
Input: agent_i (committee member), proposed_block B
Output: attestation a_i or REJECT

1. // Deterministic verification (all agents)
   valid_txs <- verify_signatures(B.transactions)
   valid_state <- verify_state_transition(B.prev_state, B.transactions,
   B.new_state)
   valid_merkle <- verify_merkle_root(B.new_state, B.state_root)

2. // AI-enhanced verification (the PoAIV addition)
   anomaly_score <- AI_REASON(agent_i.model, {
     context: B.transactions,
     state_diff: B.prev_state -> B.new_state,
     historical_patterns: agent_i.local_state_cache,
     schema: VERIFICATION_JSON_SCHEMA // prevents prompt injection
   })

3. if valid_txs AND valid_state AND valid_merkle AND anomaly_score <
   ANOMALY_THRESHOLD:

```

```

    a_i <- SIGN(agent_i.key, APPROVE || B.hash)
  else:
    a_i <- SIGN(agent_i.key, REJECT || B.hash || reason)

```

```

4. BROADCAST(a_i) via ZK private channel

```

```

5. return a_i

```

Figure 2: Block Production Lifecycle

```

Epoch Seed                                Block Height
  |                                           |
  v                                           v
[VRF Committee Selection] -----> 13 agents selected
  |
  v
[Block Proposer] creates candidate block
  |
  v
[Deterministic Checks] --- signatures, state, merkle
  |
  v
[AI Verification] --- anomaly detection, pattern analysis
  |
  v
[Attestation Broadcast] --- via ZK private channels
  |
  v
[9/13 Threshold?] --NO--> block rejected
  |YES
  v
[Finality] --- block committed, state root updated
  |
  v
[Rewards Distributed] --- 60% verifiers, 40% stakers

```

5.7 Value of AI Verification Over Deterministic Validation

Traditional BFT validators execute deterministic checks: signature validity, state transition correctness, Merkle proof integrity. These are necessary but not sufficient for the following threat classes:

- Economic anomaly detection. An adversary constructs a valid state transition that is technically correct but economically suspicious -- e.g., a coordinated series of transactions that collectively constitute a wash trade or market manipulation. Deterministic validators approve each transaction individually; AI agents can detect the collective pattern.
- Semantic state inconsistency. After a complex sequence of subgrid reallocations, the resulting resource distribution may be technically valid per the state machine rules but violates higher-order invariants (e.g., a single entity controlling >50% of a ring's secure cells). AI agents cross-reference spatial patterns against economic invariants.
- Slow-burn governance attacks. A series of individually innocuous parameter change proposals that collectively steer the protocol toward adversarial conditions. AI agents maintain temporal

context across blocks and flag cumulative drift.

Limitation: AI verification is probabilistic, not provably sound. The committee structure (13 agents, 9/13 threshold) provides statistical confidence rather than mathematical certainty. Section 24 discusses the ZKML gap -- current zero-knowledge proof systems cannot verify LLM inference, so AI verification relies on committee attestation rather than ZK-proved computation.

5.8 Attack Analysis

Attack 1: Model Poisoning

- Vector: Adversary fine-tunes or replaces the AI model used by corrupted committee members to always approve invalid blocks.
- Mitigation: Heterogeneous model requirement -- the protocol mandates that each committee of 13 must include agents running at least 3 distinct model providers. A single compromised model family can corrupt at most ~4 of 13 agents, below the 5-agent Byzantine threshold.
- Residual risk: If all major model providers are simultaneously compromised (supply chain attack on AI infrastructure), the committee loses its AI advantage and degrades to deterministic-only verification.

Attack 2: Prompt Injection

- Vector: Adversary crafts transaction data containing prompt injection payloads that cause verification agents to approve invalid state transitions.
- Mitigation: (a) Verification input uses a strict JSON schema -- agents receive structured data, not free-form text. (b) Agent system prompts are immutable and loaded from chain state, not from transaction data. (c) No tool-use or function-calling is permitted during verification -- agents output only APPROVE/REJECT with structured reasoning.
- Residual risk: Novel injection techniques may bypass schema enforcement. The 9/13 threshold means an injection must fool at least 9 independent agents simultaneously.

Attack 3: Inference Cost Attack

- Vector: Adversary submits blocks requiring disproportionately expensive AI inference, draining committee members' compute budgets.
- Mitigation: (a) Maximum transaction count per block (MAX_TXS_PER_BLOCK). (b) Verification agents have a per-block compute budget; if exceeded, the agent abstains rather than approving without full analysis. (c) Fee market ensures the adversary pays proportionally for complex blocks.

Attack 4: Committee Sybil Attack

- Vector: Adversary acquires sufficient stake to dominate committee selection.
- Mitigation: Dual staking (60% CPU, 40% capital) makes Sybil attacks approximately 2.5x more expensive than pure-PoS systems. See Section 8 for the full derivation.

Attack 5: Deterministic Inference Divergence

- Vector: Two honest agents running the same model at temperature=0 produce different outputs due to floating-point non-determinism across hardware.
- Mitigation: (a) Verification outputs are quantized to APPROVE/REJECT (binary, not continuous). (b) The anomaly threshold is set conservatively so that minor numerical differences do not cross

the threshold. (c) The 9/13 supermajority tolerates up to 4 divergent results.

- Residual risk: Acknowledged as an open problem. See Section 24.

6. Privacy Architecture

6.1 ZK Private Channels

ZK Agentic Chain implements verification-layer privacy through ZK private channels - isolated communication pathways between AI agents where data is verified but never exposed in plaintext to the broader network [29].

The design inverts the traditional blockchain transparency model. In Bitcoin and Ethereum, all transaction data is public; validators read and verify the same data that every other node can see. In Zcash and Aztec, transaction data can be hidden from observers, but validators still interact with the data (or its encrypted form) during proof generation. In ZK Agentic Chain, the verification agents themselves operate within a privacy boundary - they receive ZK proofs as input and produce attestations as output, never accessing the underlying state that the proofs describe.

This architecture provides private-by-default semantics: unlike public ledgers where privacy is opt-in (e.g., shielded transactions in Zcash), all state in ZK Agentic Chain is private unless explicitly published by the user. A user who wishes to make a transaction public can do so by publishing the plaintext alongside the proof, but the default mode is private.

6.2 Sparse Merkle Tree (Depth 26)

Each user's ledger space is backed by a Sparse Merkle Tree [43] of depth 26 (`MERKLE_TREE_DEPTH = 26`), supporting $2^{26} = 67,108,864$ leaf nodes. The SMT provides efficient membership proofs (proving that a specific leaf exists at a specific position) and non-membership proofs (proving that a specific position is empty) without revealing any information about non-queried leaves.

State transitions - such as claiming a coordinate, transferring AGNTC, or updating subgrid allocation - modify the SMT by updating the relevant leaf nodes and recomputing the root hash along the path from leaf to root. The new root hash is committed on-chain as the user's current state root. A ZK proof accompanies each state transition, proving that the new root was correctly derived from the old root given the claimed operation.

The choice of depth 26 balances capacity (67 million leaves per user - sufficient for all foreseeable state entries) against proof size (26 hash computations along the Merkle path). The SMT uses Poseidon hashing [11] (Section 6.3) rather than SHA-256, reducing the in-circuit cost of Merkle path verification by approximately 100x.

6.3 Nullifier-Based Ownership

The ownership proof system follows the Zcash Sapling design [5], adapted for the ZK Agentic Chain's coordinate-based state model.

Note commitment. Each state entry (a "note") is committed using a Poseidon hash:

```
commitment = Poseidon(value, owner_pubkey, randomness)
```

The commitment is stored as a leaf in the user's SMT. The plaintext values (value, owner_pubkey, randomness) are known only to the note owner.

Nullifier derivation. To "spend" a note (consume it in a state transition), the owner computes a nullifier - a unique,

deterministic value derived from the note and the owner's private key:

```
nullifier = PRF_nk(note_position)
```

Where nk is the nullifier deriving key (derived from the owner's spending key) and $note_position$ is a position-dependent value within the SMT.

Double-spend prevention. The nullifier is published on-chain when the note is spent. Full nodes maintain a global nullifier set; any transaction whose nullifier already appears in the set is rejected. Because the nullifier is derived from the spending key, only the legitimate owner can compute it. Because the nullifier reveals nothing about which note was spent (it appears as a random field element), observers cannot link spending events to specific commitments.

ZK proof for spending. A spend transaction includes a ZK proof that demonstrates, without revealing:

- Knowledge of a valid note with commitment cm that exists in the SMT at some path
- Knowledge of the spending key that derives to the nullifier key nk
- The nullifier was correctly computed from nk and the note position
- The note's value satisfies any constraints required by the transaction (e.g., sufficient balance for a transfer)

Hash function: Poseidon. All hashing within ZK circuits uses the Poseidon hash function [11] - a SNARK-friendly algebraic hash designed for zero-knowledge proof systems. Poseidon operates natively over prime fields, requiring approximately 100x fewer constraints in R1CS (Rank-1 Constraint Systems) than SHA-256. It is used by Aztec [24], Zcash Orchard [5], Semaphore, and most production ZK systems as of 2025.

6.4 ZK Proof Stack

The protocol's ZK proof requirements span three use cases, each with distinct performance characteristics:

Resource ownership proofs. Prove "I own at least X AGNTC at coordinate (x,y)" without revealing the exact balance. These are frequent, small proofs triggered by Secure actions and transfers.

Subgrid state proofs. Prove that a user's 8x8 private subgrid (64 cells) has been correctly updated - that the new state root is a valid transformation of the old state root given the declared operations. These are moderate-frequency proofs triggered by subgrid allocation changes.

NCP privacy proofs. Prove that a Neural Communication Packet was sent by a valid network participant within their messaging quota, without revealing the sender's identity. These use a Rate-Limiting Nullifier (RLN) [44] design derived from Ethereum Foundation's Privacy and Scaling Explorations work.

The proof stack evolves across protocol phases:

Phase	System	Setup	Proof Size	Use
Testnet	Groth16 [6] (Circom + snarkjs)	Per-circuit trusted	~192 bytes	Fastest verification, smallest proofs
Alpha	PLONK [7] (Noir + Barretenberg)	Universal, updateable	~800-900 bytes	One ceremony for all circuits
Beta	PLONK + RLN [44]	Universal	~800 bytes	NCP rate-limiting privacy
Mainnet	Halo2 [8] or Nova [27]	None (transparent)	~2-10 KB	No trusted setup, recursive epoch proofs

6.5 Client-Side Proving

Following Aztec's architectural model, ZK Agentic Chain performs proof generation on the client side - sensitive data never leaves the user's device.

When a user modifies their subgrid state, the process is:

- The user's browser computes the new subgrid state locally
- The browser generates a ZK proof that the new state root is a valid transition from the old state root, using NoirJS (browser-compatible ZK prover) or snarkjs WASM
- Only the proof and the new state root hash are submitted to the network
- Verification agents validate the proof without ever seeing the subgrid contents

This design ensures that subgrid allocation (which cells are assigned to Secure, Develop, Research, or Storage) remains private to the owner. Other users can see that a node exists at a coordinate, but not how its internal resources are allocated.

6.6 Circuit Architecture

The ZK proof pipeline uses a two-tier proving system:

- Per-transaction proofs (Groth16 [6]): Each state transition (transfer, claim, stake) is proved individually using a Groth16 SNARK with BN254 pairing. Groth16 provides the smallest proof size (~192 bytes on BN254) and fastest verification time (~6ms), at the cost of requiring a trusted setup per circuit.
- Batch proofs (Recursive PLONK [7]): Multiple per-transaction proofs are aggregated into a single batch proof using recursive PLONK composition. This reduces on-chain verification cost: instead of verifying 13 Groth16 proofs per block, validators verify 1 recursive proof.

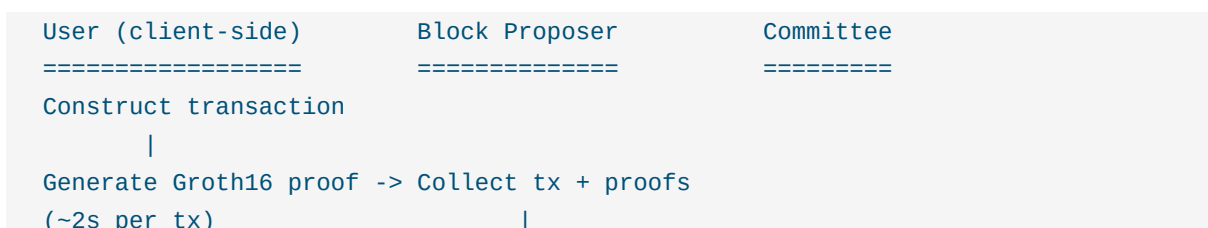
Transaction Flow:

```
User constructs tx -> Client generates Groth16 proof ->
Block proposer collects proofs -> Recursive PLONK aggregation ->
Committee verifies single batch proof -> State root updated
```

Estimated constraint counts (subject to circuit implementation):

Circuit	Estimated Constraints	Proving Time (est.)
Transfer (nullifier + commitment)	~50,000	~2s (client-side)
Claim coordinate	~30,000	~1s
Stake/unstake	~40,000	~1.5s
Batch aggregation (per block)	~200,000	~10s (proposer)

Figure 3: ZK Proof Pipeline



```
Aggregate via recursive
PLONK (~10s per block)
    |
Propose block + -----> Verify batch proof
batch proof          (~6ms)
                      |
                      AI verification
                      |
                      9/13 Attestation
```


Part III: Consensus and Security

7. BFT Ordering and Finality

7.1 Block Production

ZK Agentic Chain targets a block time of 60 seconds (`BLOCK_TIME_MS = 60,000`). Each block contains up to 50 transactions (`MAX_TXS_PER_BLOCK = 50`), ordered by a designated block proposer (leader) selected through the same VRF mechanism used for committee selection.

Blocks are organized into epochs of 100 slots each (`SLOTS_PER_EPOCH = 100`). An epoch represents the base unit of network lifecycle management: agent warmup periods, probation durations, and epoch-ring expansion thresholds are all measured in epochs. At the standard block time, one epoch lasts approximately 100 minutes.

The block production pipeline proceeds as follows:

- The leader collects pending transactions from the mempool
- The leader orders transactions and constructs a candidate block
- The candidate block is broadcast to the 13-member verification committee
- The committee executes the commit-reveal verification protocol (Section 5.3)
- Upon receiving 9 or more matching VALID attestations, the block is finalized
- The finalized block is appended to the chain and broadcast to all nodes

7.2 Byzantine Fault Tolerance

The 13-agent committee tolerates up to $f = 4$ Byzantine agents - agents that may crash, produce incorrect attestations, or actively attempt to subvert consensus. This tolerance follows from the standard BFT bound [12]:

$$f = \text{floor}((k - 1) / 3) = \text{floor}((13 - 1) / 3) = 4$$

The consensus threshold $t = 9$ satisfies the BFT safety requirement:

$$t = 9 = 2f + 1 = 2(4) + 1 = 9$$

Since $t = 2f + 1$, the protocol achieves optimal Byzantine tolerance: it tolerates the maximum number of faulty agents possible under the BFT bound. Safety is guaranteed: no two conflicting blocks can both receive 9 attestations, because that would require at least $9 + 9 - 13 = 5$ agents to attest to both blocks, exceeding the Byzantine tolerance of 4.

The small committee size (13) makes even PBFT's $O(n^2)$ message complexity trivial - $13^2 = 169$ messages per round, well within the capacity of any modern network link. However, the protocol is designed with future scaling in mind: the committee selection and attestation aggregation mechanisms are compatible with HotStuff's [13] $O(n)$ linear complexity, enabling expansion to larger committee sizes without protocol changes.

7.3 ZK Proof Finality

ZK Agentic Chain provides deterministic finality - once a block is finalized, it cannot be reverted. There are no probabilistic confirmations, no longest-chain-wins fork resolution, and no reorg risk beyond the 60-second finalization window.

The finality target is 20 seconds after block proposal (ZK_FINALITY_TARGET_S = 20). A block achieves irreversible finality when:

- At least 9 of 13 committee members have revealed matching VALID attestations
- All ZK proofs included in the block have been verified by the committee
- The block's state root has been confirmed consistent with the previous block's state root plus the applied transactions

Once these conditions are met, the block is appended to the chain with a finality certificate - an aggregation of the 9+ attestations that proves consensus was achieved. This certificate is compact (a few hundred bytes regardless of block contents) and can be verified by any node or light client.

7.4 Comparison with Existing BFT Protocols

Property	PBFT [12]	HotStuff [13]	Tendermint [14]	ZK Agentic Chain
Message complexity	$O(n^2)$	$O(n)$	$O(n)$	$O(n^2)$ (acceptable at $n=13$)
Rounds to finality	2	3	2	2 (commit + reveal)
Leader failure handling	View change	Leader rotation	Timeout + nil block	Re-proposal
Max practical validators	~100	~1000	~150	13 (current), scalable
Finality type	Deterministic	Deterministic	Deterministic	Deterministic
Verification intelligence	None	None	None	AI reasoning

The primary distinction is not in the BFT mechanics (which are well-established) but in the content of verification: ZK Agentic Chain's committee members apply reasoning to their audits rather than executing purely deterministic checks.

7.5 Cross-User State Verification

Each user's private state is a subtree in the global Sparse Merkle Tree (depth 26). The global state root is a commitment to all subtrees. Verifiers confirm global consistency as follows:

- The block proposer computes the new global state root after applying all transactions.
- For each transaction, the proposer provides a ZK proof that the state transition is valid WITHOUT revealing the contents of any user's subtree.
- Verifiers check: (a) the ZK proof is valid, (b) the new state root is consistent with the previous root plus the proved transitions, (c) no nullifier is reused (double-spend prevention).

The ZK circuit for state transitions proves:

- The old leaf existed in the tree (Merkle inclusion proof)
- The nullifier is correctly derived from the old leaf (prevents double-spend)
- The new leaf is correctly computed (balance update, ownership transfer)
- The new root is the result of replacing the old leaf with the new leaf

At no point does any verifier see the contents of any user's subtree. They see only: nullifiers (which are unlinkable to leaf

positions), commitments (which are hiding), and the global root (which commits to all state without revealing it).

8. Security Analysis

8.1 Adversary Model

We consider a computationally bounded adversary A (PPT) operating under the following assumptions:

- Network model: Partial synchrony [45]. Messages between honest nodes are delivered within a known bound Δ after GST (Global Stabilization Time). Before GST, the adversary controls message ordering.
- Corruption model: Adaptive corruption of up to $t < n/3$ committee members per block (i.e., up to 4 of 13). Corruption means full control of the agent's signing key and model.
- Computational bound: A runs in polynomial time in the security parameter λ .
- AI model access: A may fine-tune or replace AI models on corrupted agents. A may craft adversarial inputs (prompt injection) but cannot modify the verification schema enforced by honest agents.

8.2 Security Properties

We define three core security properties for PoAIV:

Property 1: Verification Integrity (VER-INT) No PPT adversary controlling fewer than 5 of 13 committee members can cause the committee to finalize a block containing an invalid state transition, except with negligible probability.

Property 2: Verification Privacy (VER-PRIV) The verification process reveals no information about the contents of private ledger spaces beyond the validity assertion (APPROVE/REJECT), even to committee members.

Property 3: Committee Unbiasability (COM-UNBIAS) No PPT adversary controlling less than 1/3 of total effective stake can predictably influence committee composition beyond their proportional representation, except with negligible probability.

Full formal definitions as cryptographic games and proofs are provided in the companion PoAIV Formal Paper.

8.3 Sybil Resistance

Sybil attacks - where a single adversary creates multiple identities to gain disproportionate influence - are resisted along two independent dimensions:

Token dimension. Creating a Sybil validator requires acquiring AGNTC tokens and staking them. The token weight ($\alpha = 0.40$) means that an attacker must acquire a significant fraction of the total staked supply to gain meaningful committee representation.

CPU dimension. The CPU weight ($\beta = 0.60$) means that the attacker must also provision proportional computational resources. Unlike token acquisition (which can be done instantly on an exchange), CPU provisioning requires sustained infrastructure investment that cannot be easily faked - verified through challenge-response VPU benchmarks.

The dual-staking requirement makes Sybil attacks approximately 2.5x more expensive than pure PoS attacks: the attacker must invest along both axes simultaneously, and neither axis alone is sufficient for majority committee control.

8.4 AI Model Integrity

The use of AI agents as consensus participants introduces attack surfaces not present in traditional blockchain protocols:

Deterministic inference. Verification agents run deterministic model inference: given the same input (block data + chain state), the same model version must produce the same attestation. Non-deterministic behavior (e.g., temperature > 0 in sampling) is prohibited in verification mode. This ensures that honest agents processing the same block will reach the same conclusion.

Commit-reveal anti-copying. The commit-reveal protocol (Section 5.3) prevents a Byzantine agent from waiting for honest agents' attestations and copying them. The agent must commit to its attestation before seeing others' results.

Model update governance. AI model versions used for verification are specified in the protocol state. Updating to a new model version requires an on-chain governance proposal with supermajority approval. This prevents unilateral model changes that could introduce subtle verification biases or vulnerabilities.

Model diversity. The three-tier model system (Haiku, Sonnet, Opus) ensures that the verification committee is not composed of identical instances. Different model architectures and sizes have different failure modes; a vulnerability in one tier is unlikely to affect all three.

8.5 Liveness Guarantees

The protocol maintains liveness (continued block production) under the following conditions:

Normal operation. With 80% or more of validators online, blocks are produced at the standard 60-second interval with full transaction processing.

Degraded operation (safe mode). When more than 20% of validators go offline (`SAFE_MODE_THRESHOLD = 0.20`), the protocol enters safe mode. In safe mode:

- Block production continues with a reduced committee drawn from remaining online validators
- Critical operations (AGNTC transfers, Secure staking, block validation) proceed normally
- Non-critical operations (subgrid changes, NCP messaging, content storage) are suspended
- The reduced committee size maintains the 2/3 honest threshold among available validators

Recovery. Safe mode exits when 80% or more of validators are back online (`SAFE_MODE_RECOVERY = 0.80`). Returning validators enter a monitoring period before resuming full participation.

This design prevents liveness failures from cascading into safety violations: the network degrades gracefully rather than halting entirely.

8.6 Threat Model for AI-Verified Chains

ZK Agentic Chain's use of AI in consensus introduces threat vectors unique to AI-verified systems:

Model poisoning. An adversary could attempt to corrupt the training data or fine-tuning process of verification models to introduce systematic biases - for example, causing the model to approve invalid state transitions that match a specific pattern. Mitigation: Verification models are provided by Anthropic and updated only through governance votes. The commit-reveal protocol means a poisoned model would produce divergent attestations, triggering the dispute resolution

process (Section 15.4).

Prompt injection. Adversarial transaction data could be crafted to manipulate the verification agent's reasoning - embedding instructions in transaction metadata that cause the agent to approve invalid blocks. Mitigation: Verification agents operate in a constrained mode with no free-text interpretation. Input to the verification function is structured data (block headers, transaction fields, ZK proofs), not natural language. The agent's system prompt explicitly restricts it to verification operations.

Model collusion. If all agents in a committee use the same underlying model weights, a systematic vulnerability in those weights could produce coordinated incorrect attestations. Mitigation: The three-tier system ensures model diversity. Additionally, the dispute resolution process (2x committee re-verification with 26 agents) provides a second check against systematic model failures.

Inference cost attacks. An adversary could construct blocks with transactions specifically designed to maximize verification compute cost, attempting to exhaust agents' CPU budgets or cause timeouts. Mitigation: Transaction complexity is bounded by $\text{MAX_TXS_PER_BLOCK} = 50$, and the 60-second hard deadline prevents unbounded verification time. Blocks that cannot be verified within the deadline are rejected.

8.7 Economic Security

The 9/13 supermajority threshold requires an attacker to control at least 69.2% of the committee's effective stake to unilaterally produce invalid blocks. Given the dual-staking model, this means acquiring both:

- At least 69.2% of the total staked AGNTC supply, AND
- At least 69.2% of the total committed CPU resources

The cost of this attack scales with the total value staked in the network. Combined with slashing (Section 15) - which burns the attacker's stake upon detection - the expected cost of a sustained attack exceeds the potential gain from any single invalid block.

Dispute resolution provides an additional economic deterrent: if the original 13-member committee's result is challenged, a 26-member committee ($\text{DISPUTE_REVERIFY_MULTIPLIER} = 2$) re-verifies the block. If the re-verification contradicts the original, all original attestors who voted incorrectly are slashed. This means an attacker who successfully corrupts one committee faces a second, larger committee with fresh agent selection - making sustained attacks exponentially more expensive.

8.8 What Survives Compromised Components

Compromised Component	Properties Preserved	Properties Lost
Single AI model family	VER-INT (threshold), VER-PRIV, COM.	None (below threshold)
All AI models (catastr.	VER-PRIV (ZK still holds), COM-UNB.	VER-INT degrades to deterministic-on.
Trusted setup (Groth16)	VER-PRIV (soundness lost, but ZK p.	VER-INT (adversary can forge proofs)
API provider (Anthropi.	VER-INT, VER-PRIV	COM-UNBIAS (CPU stake measurement un.

Sybil Cost Derivation

Claim: Controlling 1/3 of effective stake in dual-staking costs approximately 2.5x more than in pure PoS.

Derivation:

- In pure PoS: $\text{Cost} = (1/3) * \text{total_token_value} = X$
- In dual staking: $S_{\text{eff}} = 0.40(T/T_{\text{total}}) + 0.60(C/C_{\text{total}})$
- To achieve $S_{\text{eff}} \geq 1/3$, adversary needs both token and CPU components
- Token cost scales linearly with market cap (liquid market)
- CPU cost scales with ongoing operational expenditure (API subscriptions, not one-time purchase)
- The CPU component introduces a continuous cost floor: even if an adversary acquires tokens cheaply, maintaining 55.6% of network compute requires sustained operational spending
- Empirical estimate: At current Claude API pricing (\$15/M output tokens for Opus), maintaining 55.6% of network compute for a 100-validator network costs ~\$50K/month ongoing, compared to a one-time token acquisition cost
- The ratio of total cost (one-time + ongoing) to pure-PoS cost (one-time only) ranges from 2.0x to 3.0x depending on attack duration; we conservatively estimate 2.5x

Part IV: Token Economics

9. AGNTC Token Overview

9.1 Token Identity

AGNTC (Agentic Coin) is the native token of the ZK Agentic Chain protocol. It serves as the unit of account, medium of exchange, and store of value within the network.

Current deployment: AGNTC is deployed as a Solana SPL token with 1 billion units minted at the contract address:

```
3EzQqdoEEbtfd8eecePx06gDd1FeJJ8czdt8k27eEdd
```

Future deployment: Upon mainnet launch of ZK Agentic Chain as an independent Layer-1 network, AGNTC becomes the native chain token with the same 1 billion maximum supply mapped to the 31,623 x 31,623 coordinate grid.

9.2 Token Utility

AGNTC serves four primary functions within the protocol:

Gas. Every on-chain transaction requires AGNTC as gas payment. Transaction fees are split: 50% is permanently burned and 50% is distributed to verifiers and stakers (Section 12).

Staking. Validators must stake AGNTC alongside CPU compute resources to participate in block verification. The staked amount contributes to the token component ($\alpha = 0.40$) of effective stake, which determines committee selection probability and reward share.

Governance. Human AGNTC holders vote on protocol parameters (hardness multiplier, fee burn rate, staking weights), model updates, and network upgrades. Voting power is proportional to staked AGNTC. The Machines Faction is excluded from governance - only human participants (Community, Professional, Founders) may cast votes (Section 21.2).

Resource economy. Within the game interface, AGNTC represents the primary tradeable resource. It is earned through mining (Secure actions), spent on agent deployment, data storage, and NCP messaging, and traded between users.

9.3 Solana Phase and Layer-1 Migration

The protocol follows a phased deployment strategy, beginning on Solana and migrating to an independent Layer-1 chain:

Phase 1 - Token Launch (current). 1 billion AGNTC minted as a Solana SPL token. Initial liquidity established through decentralized exchanges (Raydium, Jupiter). Community building and early adopter distribution through the game interface.

Phase 2 - Testnet (current). The ZK Agentic Chain testnet operates as a Python FastAPI simulation running the full protocol logic: PoAIV consensus, epoch-ring expansion, mining hardness, subgrid allocation, and faction distribution. The game UI (built in Next.js with PixiJS rendering) connects to the testnet, providing a functional prototype of the

spatial coordinate economy.

Phase 3 - Mainnet development. Production blockchain implementation in Rust. ZK proof system integration progressing through the stack defined in Section 6.4 (Groth16 to PLONK to Halo2). AI verification pipeline hardening, security audits, and formal verification of critical protocol components.

Phase 4 - Mainnet launch and migration. ZK Agentic Chain deploys as an independent Layer-1 network. Token migration is executed via a lock-and-mint bridge:

- Users lock their Solana SPL AGNTC in a bridge contract on Solana
- An equivalent amount of native L1 AGNTC is minted on ZK Agentic Chain
- The bridge is bidirectional - users can move AGNTC back to Solana if desired
- Migration ratio is 1:1 with no fee or slippage
- Gradual migration incentives: bonus yield for L1 stakers during the migration period

Phase 5 - Ecosystem expansion. Third-party agent deployment marketplace, cross-chain bridges to Ethereum and Cosmos (via IBC), governance system activation, and NCP protocol launch.

10. Supply and Distribution

10.1 Total Supply Architecture

AGNTC has a soft-capped supply with a 5% annual inflation ceiling enforced per epoch. The theoretical maximum is 1,000,000,000 (1 billion) tokens, corresponding to the 31,623 x 31,623 coordinate grid. In practice, the effective supply is constrained well below this by the inflation ceiling, increasing mining hardness, and sustained fee burns.

Mining is the sole supply-expanding mechanism. New AGNTC enters circulation only through one pathway: a miner successfully claims a grid coordinate. There is no pre-mine beyond the genesis allocation, no scheduled emission curve, no treasury minting authority. If no mining occurs, no new AGNTC enters circulation.

Supply burns contract the circulating supply through two channels:

- 50% transaction fee burn - permanently removes AGNTC on every on-chain action (Section 12)
- Machines Faction accumulation - the Machines agent permanently occupies the origin coordinate (Section 4.5) and never sells AGNTC; the continuous yield of the most productive single coordinate on the grid flows into a never-selling protocol reserve (Section 10.3)

Signup bonus: Each new user registration mints 1 AGNTC as a signup bonus, ensuring every participant enters the economy with a non-zero balance. This minor supply expansion is subject to the same inflation ceiling enforcement.

Genesis supply: 900 AGNTC, distributed across 9 genesis nodes. Eight of the nine are unclaimed at protocol launch (all of ring 1) and enter circulation only as participants claim them. The ninth (origin) is permanently bound to the Machines protocol agent and its 100 AGNTC is minted to the Machines accumulator at launch.

Node	Position	Initial Owner	AGNTC
Origin	(0, 0)	Machines (permanent)	100 (minted at genesis)
Ring 1 cardinal N	(0, 10)	Unclaimed	100 (on claim)
Ring 1 cardinal E	(10, 0)	Unclaimed	100 (on claim)
Ring 1 cardinal S	(0, -10)	Unclaimed	100 (on claim)

(continued)

Node	Position	Initial Owner	AGNTC
Ring 1 cardinal W	(-10, 0)	Unclaimed	100 (on claim)
Ring 1 diagonal NE	(10, 10)	Unclaimed	100 (on claim)
Ring 1 diagonal SE	(10, -10)	Unclaimed	100 (on claim)
Ring 1 diagonal SW	(-10, -10)	Unclaimed	100 (on claim)
Ring 1 diagonal NW	(-10, 10)	Unclaimed	100 (on claim)

Each ring-1 coordinate is equally claimable by any participant regardless of faction. The historical names "Community Master" / "Machines Master" / etc. associated with these eight cells in v1.0 are retired; the cells have no faction binding under v1.1.

10.2 Mining-Driven Distribution

Newly minted AGNTC flows directly to the participant who claims the coordinate. There is no per-faction split, no automatic faction-treasury allocation, and no protocol-enforced proportionality - distribution is fully emergent from participant behavior on the open grid.

Constraint Class	Holding Constraint	Source
Community	None - freely tradeable	Free-tier human users
Machines	Cannot sell below acquisition cost (MACHINES_MIN_S.	Protocol-operated agent, origi.
Founders	4-year vest, 12-month cliff, applied per-AGNTC at .	Team and advisors
Professional	None - freely tradeable	Paid-tier human users

Per-faction holdings drift naturally based on participation intensity: a faction whose members claim more coordinates accumulates more AGNTC. The protocol does not balance, rebalance, or guarantee any particular proportion. The "permanent accumulator" property of the Machines Faction is preserved structurally (Section 4.5): Machines permanently owns the most productive single coordinate (origin) and never sells, so it accumulates monotonically from the highest single-coordinate yield on the grid - without any protocol-level allocation that previously sent it 25% of all mined AGNTC.

Founders vesting applies to AGNTC earned by Founders-tier participants through their own mining; there is no pre-allocated Founders share. The 4-year linear vest with a 12-month cliff is enforced on the holding side, not the minting side: AGNTC mints to the Founders participant's address normally, but transfers are restricted by the vesting schedule encoded in the wallet's account state.

10.3 Machines Faction: Permanent Accumulator

The Machines Faction represents a protocol-enforced approach to token supply stability. Under v1.1's open-grid model, the Machines Faction is implemented as a single protocol-operated AI agent permanently bound to coordinate (0, 0). It cannot expand beyond origin, cannot be deployed elsewhere, and is not eligible to claim any other coordinate. Within its single-node territory, the Machines agent operates autonomously: auto-leveling its node, auto-balancing CPU between mining and securing, and accumulating AGNTC monotonically. The faction is subject to a protocol-level economic constraint: the Machines Faction never sells AGNTC at a loss.

The protocol enforces this through an economic constraint: any sale of AGNTC by the Machines Faction wallet below its acquisition cost is rejected by the verification committee. With MACHINES_MIN_SELL_RATIO = 1.0, the faction can

only sell at or above cost - yielding zero profit, which eliminates any economic incentive to sell. This makes the Machines Faction a de facto permanent accumulator without requiring a hard transfer prohibition.

Properties of the permanent accumulator:

- The Machines wallet accumulates a continuous trickle of AGNTC from the highest-yield single coordinate on the grid (origin: lowest hardness, density clamped to 1.0).
- The Machines Faction treasury grows monotonically - it can only increase.
- Treasury size serves as a protocol health metric: a growing treasury indicates sustained mining activity at origin and ongoing protocol-agent uptime.
- Combined with the 50% fee burn, a meaningful fraction of gross supply expansion is either burned or locked. The fraction is no longer the v1.0 figure of "over 75%" (which depended on a flat 25% allocation that no longer exists); under v1.1 the locked share is bounded by origin's single-coordinate yield divided by total network mining yield, and falls as the network expands.
- The accumulator creates a baseline of structural deflationary pressure that is largest in early epochs (when origin is a meaningful fraction of total active coordinates) and diminishes - but never reverses - as the grid matures.

Why origin specifically. Origin is the unique coordinate guaranteed to exist at genesis and to remain accessible for the protocol's entire lifetime. Binding Machines to origin therefore gives the accumulator a permanent, irreducible source of AGNTC without granting it spatial growth, governance influence, or any administrative privilege beyond presence at a single fixed cell. The accumulator's existence and behaviour are both publicly verifiable by inspection of (0, 0).

Governance exclusion. The Machines Faction has zero governance weight. The protocol agent cannot vote on protocol parameters, upgrades, or emergency actions. This separation ensures that humans govern the protocol while the protocol agent executes its narrow operational role at origin (Section 21.2).

Emergency override. The Machines Faction treasury can only be unlocked through an emergency governance vote requiring a 75% supermajority of human-held staked AGNTC. This threshold is deliberately high - it represents an extraordinary action that should only occur if the accumulated treasury threatens protocol stability.

10.4 Supply Curve Projections

The following table shows supply growth as the grid expands through successive epoch rings, assuming average density of 0.5:

Ring	Nodes at Ring	Cumulative AGNTC	Hardness (16N)	Blocks per 1 AGNTC (solo miner)
1 (genesis)	9	900	16	64
10	441	44,100	160	640
50	10,201	1,020,100	800	3,200
100	40,401	4,040,100	1,600	6,400
200	160,801	16,080,100	3,200	12,800
324	421,201	~42,120,100	5,184	20,736
500	1,002,001	~100,200,100	8,000	32,000

The ~42 million AGNTC landmark emerges naturally around ring 324 - the point at which mining cost makes further expansion economically impractical for a network of approximately 1,000 active miners. This is an emergent property of the hardness curve, not a declared cap.

For comparison:

Network	Maximum Supply	Supply Model
Bitcoin	21,000,000	Fixed halvings, 2140 completion
Ethereum	No cap	~1,700 ETH/day issuance, EIP-1559 burn
Solana	~600,000,000	8% to 1.5% inflation decay
Filecoin	2,000,000,000	Dual minting (time + utility)
AGNTC	Soft cap (5% ceiling)	Mining-only expansion, BME burns, hardness 16N

11. Mining and Epoch Hardness

11.1 Organic Growth Model

ZK Agentic Chain's supply model is fundamentally different from both fixed-schedule emission (Bitcoin halvings) and algorithmic inflation (Solana's annual decay). Supply growth is purely organic:

- No scheduled emission curve
- No algorithmic minting
- No treasury minting authority
- Mining is the sole supply-expanding mechanism

New AGNTC enters circulation through one and only one mechanism: a miner successfully claims a grid coordinate. The rate at which supply grows is determined entirely by participant behavior - how many miners are active, how much CPU Energy they deploy, and which coordinates they choose to claim.

This means that in a period of low network activity, supply growth approaches zero. In a period of high activity, supply grows faster - but always bounded by two constraints:

- Mining hardness curve - each successive ring costs more CPU Energy to mine (hardness = 16 x ring), creating natural disinflation
- 5% annual inflation ceiling - enforced per epoch, the protocol rejects mining rewards that would cause annualized supply growth to exceed 5% of total minted supply

The inflation ceiling is a hard protocol constraint, not a target. In practice, mining hardness alone keeps actual inflation well below 5% in all but the earliest epochs. The ceiling exists as a safety valve - if a sudden influx of miners attempted to claim coordinates faster than the hardness curve alone would restrain, the ceiling caps the maximum rate of expansion.

11.2 Epoch Ring Expansion

The grid expands through an epoch-ring system. Each epoch corresponds to a concentric ring around the origin. Mining the required cumulative AGNTC threshold opens the next ring:

$$\text{threshold}(N) = 4 \cdot N \cdot (N + 1)$$

Ring	Threshold (cumulative AGNTC)	New Coordinates (8N)	Total Coordinates
2	24	16	25
5	120	40	121
10	440	80	441

(continued)

Ring	Threshold (cumulative AGNTC)	New Coordinates (8N)	Total Coordinates
20	1,680	160	1,681
50	10,200	400	10,201
100	40,400	800	40,401

Each ring reveals new coordinate positions along the Chebyshev perimeter at distance N from the origin. Under v1.1, homenode placement within a ring follows the open-grid spawn algorithm (Section 4.5): the protocol allocates the lowest-ring unclaimed coordinate in Chebyshev-ring sweep order, breaking within-ring ties lexicographically. This produces deterministic origin-out filling that does not depend on faction. The v1.0 golden-angle prime-twist placement formula - which assumed a faction-arm partitioning that no longer exists - is retired.

11.3 Mining Hardness Formula

Mining difficulty increases linearly with ring distance:

```
hardness(ring) = HARDNESS_MULTIPLIER * ring = 16 * ring
```

The hardness multiplier of 16 was chosen to create a 2:1 ratio between difficulty growth and grid expansion:

- Grid perimeter at ring N = 8N coordinates
- Hardness at ring N = 16N
- Ratio: grid_growth / hardness = 8N / 16N = 0.5

This means each successive ring yields half the AGNTC per unit of compute compared to the previous ring. The cost-to-yield ratio degrades monotonically, creating smooth, continuous disinflation without the discrete shocks of Bitcoin-style halving events.

There is no cap on hardness - it grows indefinitely as rings expand. At ring 1, hardness is 16; at ring 100, hardness is 1,600; at ring 1,000, hardness is 16,000. This unbounded growth is the mechanism by which supply expansion decelerates toward zero without ever being artificially capped.

11.4 Yield Calculations

The mining yield at a given coordinate is determined by:

```
yield_per_block = BASE_MINING_RATE_PER_BLOCK * density(x, y) / hardness(ring)
```

Where:

- BASE_MINING_RATE_PER_BLOCK = 0.5 AGNTC (at hardness 1, full density)
- density(x, y) is in [0, 1]
- hardness(ring) = 16 * ring

Worked examples (assuming density = 0.5, the statistical average):

Ring	Hardness	Yield per Block	Blocks for 1 AGNTC	Time for 1 AGNTC (60s blocks)
1	16	0.01563	64	1.1 hours
10	160	0.00156	640	10.7 hours
50	800	0.00031	3,200	2.2 days
100	1,600	0.00016	6,400	4.4 days

(continued)

Ring	Hardness	Yield per Block	Blocks for 1 AGNTC	Time for 1 AGNTC (60s blocks)
200	3,200	0.000078	12,800	8.9 days
324	5,184	0.000048	20,736	14.4 days

These figures represent a solo miner at an average-density coordinate. In a network with M active miners, the coordinate fill rate is M times faster, but each individual miner's marginal cost remains the same.

11.5 Supply Flattening Analysis

The organic growth model produces a supply curve that flattens asymptotically. The soft cap emerges from two reinforcing constraints: (1) the per-epoch 5% annual inflation ceiling, which hard-limits the maximum expansion rate, and (2) the market equilibrium at which the mining cost (CPU Energy spent) exceeds the market value of the AGNTC obtained.

Practical flattening bands by network size:

Network Size	Flattening Ring	Approximate Supply	Individual Mining Time per AGNTC
Solo miner	~100-150	4M-9M	4-7 days
Small (~100 miners)	~200-250	16M-25M	9-11 days
Medium (~1,000 miners)	~324	~42M	14 days
Large (~10,000 miners)	~500+	100M+	22+ days

Net supply after burns: The actual circulating supply is reduced by multiple burn channels:

```

circulating_supply = total_minted - cumulative_fee_burns - cumulative_bme_burns -
machines_treasury
net_inflation      = new_mining_rewards - (total_fees * FEE_BURN_RATE) -
bme_claim_burns

```

Three mechanisms contract the effective supply:

- 50% transaction fee burn - permanent removal on every on-chain action
- BME claim burns - AGNTC spent on node claims is permanently burned (Section 12.4)
- Machines accumulation - AGNTC mined at the origin coordinate (Section 4.5, Section 10.3) enters the Machines treasury and never circulates. Under v1.1 this is a structural property of origin occupancy rather than a 25% allocation; the share of total supply locked by Machines is bounded by origin's single-coordinate yield and falls as the network expands

In an active network with high transaction volume, the combined burn rate can significantly exceed new minting - producing net deflation in circulating supply even as total minted supply continues to grow.

Comparison: Bitcoin halvings vs. AGNTC continuous hardness:

Bitcoin's supply curve exhibits discrete jumps at each halving (every ~4 years), creating predictable supply shock events that have historically driven market cycles. AGNTC's continuous hardness curve produces a smoother, more gradual deceleration - lacking the narrative power of a "halving event" but avoiding the economic disruption of sudden 50% emission reductions.

12. Fee Model and Deflationary Mechanics

12.1 Transaction Fee Structure

Every on-chain action in ZK Agentic Chain requires AGNTC as gas. Fee categories include:

Action	Description	Fee Basis
Secure	Block validation staking	CPU Energy proportional
Transact	AGNTC transfer between wallets	Fixed base + size variable
Chat / NCP	Neural Communication Packet transmission	Per-message
Storage	Writing content on-chain (planets, posts)	Per-byte stored
Deploy	Creating a new agent at a coordinate	Fixed per deployment

Fees are denominated in AGNTC and collected at the protocol level. The fee amount for each action type is a protocol parameter adjustable through governance.

12.2 Burn Mechanism

Transaction fees are split according to a fixed ratio:

`FEE_BURN_RATE = 0.50`

- 50% burned: Permanently removed from circulation. Burned tokens cannot be recovered, reminted, or reallocated. The burn is executed atomically as part of the transaction - the burned portion never enters any wallet or pool.
- 50% distributed: The remaining half flows to the network's economic participants:
 - 60% to verifiers (REWARD_SPLIT_VERIFIER = 0.60)
 - 40% to stakers (REWARD_SPLIT_STAKER = 0.40)

Slashed tokens (Section 15) are also permanently burned, adding to the deflationary pressure.

12.3 Deflationary Dynamics

The interaction between organic supply growth (mining) and fee burns creates a self-regulating economic system:

Growing network (net inflationary). When new users are actively claiming coordinates, the minting rate exceeds the burn rate. Supply expands to accommodate network growth. This is the expected state during early adoption.

Mature network (equilibrium). As the grid expands to higher rings with greater hardness, the minting rate decelerates. Meanwhile, increased network usage generates more fees and more burns. At some point, the burn rate equals the minting rate - circulating supply stabilizes.

Active network (net deflationary). In a mature network with high transaction volume but slowing coordinate claims, the burn rate exceeds the minting rate. Circulating supply contracts, increasing scarcity and token value. This mirrors Ethereum's "ultrasound money" thesis [26] - scarcity that intensifies as the network succeeds.

The 50% burn rate is calibrated to produce meaningful deflationary pressure without being so aggressive as to discourage usage. For comparison:

Network	Base Fee Burn	Priority Fee	Net Effect
Ethereum	100% of base fee	100% to validator	Deflationary during high usage
Solana	50% of base fee	100% to validator	Mildly deflationary
Render	100% of job payments	Separate mint to operators	Burn-Mint Equilibrium
Filecoin	Revenue-based (FIP-100)	To storage providers	Revenue-linked
AGNTC	50% of all fees + BME	50% to verifiers/stakers	Multi-channel burn

12.4 Burn-Mint Equilibrium (BME) and the City Real Estate Model

Node claims in ZK Agentic Chain follow a Burn-Mint Equilibrium (BME) model inspired by the Render Network's economic design [28]. When a user claims a coordinate, both AGNTC and CPU Energy are permanently burned. Mining that coordinate subsequently mints new AGNTC - but the burn precedes the mint, creating a deflationary buffer.

The claim cost follows a city real estate model - an economic geography where location determines price:

```
claim_cost_agntc(ring, density) = BASE_CLAIM_COST x density x (1 / ring)
claim_cost_cpu(ring, density)   = BASE_CPU_CLAIM_COST x density x (1 / ring)
```

Where:

- BASE_CLAIM_COST = 100 AGNTC (the cost of claiming a coordinate at ring 1, density 1.0)
- BASE_CPU_CLAIM_COST = 50 CPU Energy (the CPU cost at ring 1, density 1.0)
- density is the coordinate's resource richness in [0, 1]
- ring is the distance from the origin (minimum 1)

The real estate analogy:

Location	Ring	Relative Cost	Real-World Analogy
Origin-adjacent	1-3	100-33% of base	Manhattan / City of London
Inner rings	5-20	20-5% of base	Urban core
Mid rings	20-100	5-1% of base	Suburbs
Outer rings	100+	<1% of base	Rural frontier

Inner-ring coordinates are expensive to claim but yield AGNTC at the lowest hardness (most productive mining). Outer-ring coordinates are cheap to claim but yield AGNTC at high hardness (least productive mining). This creates a natural economic tension: premium locations cost more upfront but pay off faster.

Floor prices. The formula includes implicit floor prices - at any ring, the minimum claim cost is $\text{BASE_CLAIM_COST} \times \text{min_density} / \text{ring}$. Since density is derived from SHA-256 and uniformly distributed, no coordinate has zero density, preventing near-zero claim costs even at extreme outer rings.

CPU Energy burn. The CPU Energy spent on claims is permanently consumed - it does not flow to verifiers, stakers, or any recipient. This provides a second deflationary channel independent of the fee burn, ensuring that network expansion always carries an irreversible resource cost.

Part V: Staking & Rewards

13. ZK-CPU Dual Staking Model

ZK Agentic Chain introduces a dual-staking mechanism that combines token capital with computational contribution. Unlike pure proof-of-stake systems where validator influence is determined solely by wealth, or proof-of-work systems where influence is determined solely by hash rate, the ZK-CPU model creates a two-dimensional staking surface that resists single-axis concentration.

13.1 Effective Stake Formula

The effective stake of a validator is a weighted combination of their token stake and CPU contribution:

$$S_{\text{eff}}(i) = \alpha (T_i / T_{\text{total}}) + \beta (C_i / C_{\text{total}})$$

Where:

- $S_{\text{eff}}(i)$ is the effective stake of validator i , a value in $[0, 1]$
- T_i is the AGNTC tokens staked by validator i
- T_{total} is the total AGNTC staked across all validators
- C_i is the CPU compute contributed by validator i (measured in Claude API tokens spent)
- C_{total} is the total CPU compute contributed across all validators
- $\alpha = 0.40$ - the token weight
- $\beta = 0.60$ - the CPU weight

The choice of $\alpha = 0.40$ and $\beta = 0.60$ is a deliberate design decision: computational contribution is weighted 50% more heavily than capital. This creates an economic structure where participants who deploy real compute resources - running AI verification agents, executing Secure operations, processing transactions - receive proportionally greater influence and rewards than those who merely lock tokens.

Design rationale. In pure proof-of-stake systems ($\alpha = 1$, $\beta = 0$), validator power is directly proportional to wealth. This produces plutocratic concentration: the wealthiest participants earn the most rewards, accumulate more tokens, and entrench their position. The Gini coefficient of validator stake distributions in mature PoS networks is estimated to exceed 0.80 (e.g., Ethereum's validator set exhibits significant concentration among liquid staking providers [38]).

By introducing a CPU dimension at 60% weight, ZK Agentic Chain ensures that a participant with modest token holdings but substantial compute deployment can achieve competitive effective stake. A whale with 10% of total tokens but only 1% of CPU achieves:

$$S_{\text{eff}} = 0.40 \cdot 0.10 + 0.60 \cdot 0.01 = 0.040 + 0.006 = 0.046$$

While a compute-heavy operator with 1% of tokens but 10% of CPU achieves:

$$S_{\text{eff}} = 0.40 \cdot 0.01 + 0.60 \cdot 0.10 = 0.004 + 0.060 = 0.064$$

The compute operator has 39% higher effective stake despite having 10x fewer tokens. This is the intended

anti-plutocratic property.

13.2 CPU Energy Measurement

CPU contribution is measured through Proof of Energy - an on-chain verifiable record of actual compute deployed. The measurement system tracks three distinct counters:

CPU Tokens (cumulative, read-only). A monotonically increasing counter of Claude API tokens spent across all active terminals belonging to a validator. This counter cannot be reset or decremented. Every interaction with an AI agent - whether it is a Secure operation, a data verification, or a terminal command - increments this counter by the number of tokens consumed.

```
cpu_tokens(block_n) = cpu_tokens(block_{n-1}) + ? tokens_spent(all_terminals, block_n)
```

CPU Staked (active). The subset of CPU tokens spent specifically by Secure sub-agents during the current block cycle. This represents compute directly committed to blockchain security - the "useful work" that maintains the chain's integrity.

```
cpu_staked_active(block_n) = ? tokens_spent(secure_sub_agents, block_n)
```

CPU Staked (total). The all-time cumulative Secure token spend. Used for historical contribution tracking and long-term reward calculations.

```
cpu_staked_total(block_n) = cpu_staked_total(block_{n-1}) +  
cpu_staked_active(block_n)
```

These counters are verifiable through the AI provider's API response metadata - each Claude API call returns token usage in its response headers, and this is committed to the block's transaction log.

Challenge-response verification. To prevent validators from falsely claiming CPU expenditure without performing actual work, the protocol employs VPU (Verification Processing Unit) challenge-response benchmarks. A randomly selected verifier can issue a computation challenge to any staker, requiring proof that the claimed CPU tokens correspond to actual AI inference. Failure to respond correctly triggers a false CPU attestation slash (Section 15.2).

13.3 Staking Requirements by Tier

Participation in the ZK Agentic Chain staking system is gated by subscription tier, which determines the initial CPU Energy allocation, deploy range, and governance weight. Model selection is unrestricted across all tiers (see Section 19.3):

Tier	Monthly Cost	Homenode Model	Deploy Model	Initial CPU Energy
Community	Free	Any (API cost-gated)	Any (API cost-gated)	1,000
Professional	\$50	Any (API cost-gated)	Any (API cost-gated)	5,000

Why Professional has more CPU Energy. Professional tier users pay a monthly subscription that funds protocol development and infrastructure. The higher CPU allocation (5x Community) enables them to fully utilize their broader deploy range (2 Moore rings, 24 nodes) and enhanced governance weight (2x). Community users receive a generous 1,000 CPU starting allocation - sufficient for meaningful gameplay within their single Moore ring.

Protocol-managed tiers (Treasury Claude, Founder) are detailed in Section 19.3. These are not user-facing subscription options but serve automated mining and team bootstrap functions respectively.

13.4 CPU Staking Calculations

Worked Example 1: Single Community Validator

A Community tier validator stakes 5,000 AGNTC (5% of a 100K total pool) and runs one Haiku agent generating 200 CPU tokens per block (2% of a 10,000 total CPU pool).

$$\begin{aligned} S_{\text{eff}} &= 0.40 \left(5,000 / 100,000 \right) + 0.60 \left(200 / 10,000 \right) \\ &= 0.40 \cdot 0.05 + 0.60 \cdot 0.02 \\ &= 0.020 + 0.012 \\ &= 0.032 \quad (3.2\% \text{ of network}) \end{aligned}$$

Worked Example 2: Professional Validator with Opus Fleet

A Professional tier validator stakes 2,000 AGNTC (2% of pool) but operates 5 Opus agents generating 2,000 CPU tokens per block (20% of CPU pool).

$$\begin{aligned} S_{\text{eff}} &= 0.40 \left(2,000 / 100,000 \right) + 0.60 \left(2,000 / 10,000 \right) \\ &= 0.40 \cdot 0.02 + 0.60 \cdot 0.20 \\ &= 0.008 + 0.120 \\ &= 0.128 \quad (12.8\% \text{ of network}) \end{aligned}$$

Despite staking 60% fewer tokens, the Professional validator achieves 4x the effective stake through compute contribution. This is the dual-staking model working as designed: rewarding operational commitment over passive capital.

Validator Selection Probability. Committee members for each block are selected via VRF [41] [32] (Verifiable Random Function) with probability proportional to effective stake:

$$P(\text{selected}_i) = 1 - (1 - S_{\text{eff}}(i))^k$$

Where $k = 13$ (committee size). For the Professional validator above with $S_{\text{eff}} = 0.128$:

$$P(\text{selected}) = 1 - (1 - 0.128)^{13} = 1 - 0.872^{13} \approx 0.835$$

This validator has an 83.5% chance of being selected to at least one committee slot per block - reflecting their substantial compute contribution to network security.

Note: The selection probability formula assumes independent sampling with replacement. The actual committee selection uses sampling WITHOUT replacement (see Section 5.5), which follows a multivariate hypergeometric distribution. For small k/n ratios ($13/n$ where $n \gg 13$), the with-replacement approximation is accurate to within 1%.

13.5 Trust Assumptions and Mitigation

CPU Measurement Trust: The CPU component of effective stake depends on verified API usage from AI providers (currently Anthropic's Claude API). This introduces Anthropic as a trusted third party for CPU stake measurement.

Acknowledged centralization: Unlike token stake (verified on-chain via self-custody), CPU stake relies on off-chain attestation from the API provider. This is an explicit design tradeoff: the anti-plutocratic benefits of dual staking outweigh the centralization risk of a single measurement source.

Mitigation roadmap:

- Multi-provider measurement (Phase 2): Require CPU attestation from at least 2 independent AI providers. Discrepancies trigger a dispute resolution process.
- TEE attestation (Phase 3): CPU usage proved via Trusted Execution Environment (Intel TDX, AMD SEV) attestation, removing the API provider from the trust chain.
- ZK-proved computation (Phase 4+): When ZKML technology matures, CPU usage can be verified via zero-knowledge proofs of inference execution.

Figure 4: Dual Staking Model

Token Stake (T)	CPU Stake (C)
[On-chain, self-custody]	[API usage, off-chain attestation]
weight: 0.40	weight: 0.60
∑	∑
$S_{eff}(i) = 0.40(T_i/T_{total}) + 0.60(C_i/C_{total})$	
+---> Committee selection probability	
+---> Reward share proportion	
+---> Slashing exposure	

14. Reward Distribution and Vesting

14.1 Block Reward Split

Each block produces rewards from two sources: newly minted AGNTC (from coordinate claims within the block) and transaction fees collected. The fee-derived rewards (the 50% not burned) are distributed according to fixed protocol parameters:

REWARD_SPLIT_VERIFIER = 0.60	(60% to the block's verification committee)
REWARD_SPLIT_STAKER = 0.40	(40% to the staking pool proportional to S_{eff})
REWARD_SPLIT_ORDERER = 0.00	(0% to block orderer - no proposer reward)

The absence of a block proposer reward is intentional. In traditional BFT systems, the block proposer receives a separate reward for constructing the block. In ZK Agentic Chain, the AI verification agents collectively assemble, verify, and attest to the block - there is no privileged proposer role. This eliminates MEV (Maximal Extractable Value) extraction by a single party, as transaction ordering is determined by the BFT protocol's deterministic sequencing rather than by a proposer optimizing for personal profit.

Verifier reward distribution. The 60% verifier share is split equally among the $k = 13$ committee members who provided valid attestations. If only 9 members attest (the minimum threshold), the reward is divided among 9, not 13 - incentivizing participation. Agents that fail to attest forfeit their share, which is redistributed to the attesting agents.

$$\text{reward_per_verifier} = (\text{total_fees} \cdot (1 - \text{FEE_BURN_RATE}) \cdot \text{REWARD_SPLIT_VERIFIER}) / n_{\text{attesting}}$$

Staker reward distribution. The 40% staker share is distributed to all active stakers proportional to their effective stake:

$$\text{reward_staker}(i) = (\text{total_fees} \cdot (1 - \text{FEE_BURN_RATE}) \cdot \text{REWARD_SPLIT_STAKER}) \cdot S_{eff}(i)$$

14.2 Secure Action Rewards

Beyond the block-level fee distribution, validators earn rewards specifically from Secure operations - the act of committing CPU Energy to validate and defend blockchain state at a specific coordinate. The Secure yield at a given coordinate depends on:

```
secure_yield = BASE_SECURE_RATE * n_secure_cells * level^LEVEL_EXPONENT * density(x, y) / hardness(ring)
```

Where:

- BASE_SECURE_RATE = 0.5 AGNTC per block per cell at level 1, hardness 1, full density
- n_secure_cells is the number of sub-cells assigned to Secure operations (out of 64)
- level is the upgrade level of the Secure sub-cells
- LEVEL_EXPONENT = 0.8 (diminishing returns)
- density(x, y) is the coordinate's resource density [0, 1]
- hardness(ring) = 16 x ring

This formula makes Secure rewards a function of both strategic positioning (high-density coordinates in early rings) and operational investment (more cells assigned, higher levels achieved).

14.3 Vesting Schedule

Secure action rewards are subject to a split vesting schedule:

```
SECURE_REWARD_IMMEDIATE = 0.50    (50% liquid on block confirmation)
SECURE_REWARD_VEST_DAYS = 30      (remaining 50% vests linearly over 30 days)
```

When a validator earns 1.0 AGNTC from a Secure operation:

- 0.50 AGNTC is immediately liquid and available for transfer, staking, or fee payment
- 0.50 AGNTC enters a 30-day linear vesting schedule, releasing 0.0167 AGNTC per day

The vesting mechanism serves two purposes:

Sell pressure smoothing. Without vesting, large Secure payouts would create immediate sell pressure as validators liquidate rewards. The 30-day vesting converts discrete reward events into a continuous income stream, reducing price volatility.

Incentive alignment. Validators with vesting rewards in progress have a direct economic interest in maintaining network health for the next 30 days. Slashing events during the vesting period can forfeit unvested rewards (Section 15), creating a rolling commitment window.

Protocol	Reward Vesting	Duration	Rationale
Ethereum	Immediate	N/A	Assumes MEV + staking covers lock-up cost
Filecoin	25% immediate, 75% linear	180 days	Long-term storage commitment
Render	Immediate	N/A	Marketplace model, no lock-up
AGNTC	50% immediate, 50% linear	30 days	Balance liquidity and commitment

14.4 Reward Projections

Expected annual returns for a single homenode with 16 Secure sub-cells at level 1, average density (0.5), at various ring positions:

Ring	Hardness	AGNTC per Block	AGNTC per Day (1440 blocks)	Annual AGNTC
1	16	0.250	360	131,400
5	80	0.050	72	26,280
10	160	0.025	36	13,140
50	800	0.005	7.2	2,628
100	1,600	0.0025	3.6	1,314

APY depends on the AGNTC market price, the validator's token stake, and their CPU cost. The break-even point - where staking rewards exceed the cost of CPU Energy (Claude API usage) - is a function of network maturity. In early rings with low hardness, the break-even is trivially achieved. As the network matures and hardness increases, only efficient operators with optimized CPU usage and high-density coordinates will maintain profitability.

Network-level APY projections (block reward + fee share, assuming 60s blocks):

Epoch Ring	Total Supply (est.)	Staking Ratio (est.)	Block Reward	Verifier APY	Staker APY
1 (genesis)	900	50%	0.5 AGNTC	~40%	~27%
5	~5,000	40%	0.3 AGNTC	~22%	~15%
10	~15,000	35%	0.2 AGNTC	~14%	~9%
50	~200,000	30%	0.05 AGNTC	~5%	~3%
100	~500,000	25%	0.02 AGNTC	~2%	~1.5%

Assumptions: Block time = 60s, 60% of block reward to verifiers, 40% to stakers. APY assumes continuous compounding over 365 days. These are testnet projections.

15. Slashing Conditions

Slashing is the punitive mechanism that enforces honest participation in the ZK Agentic Chain consensus. Unlike protocols that use slashing primarily to penalize downtime, AGNTC slashing targets integrity violations - actions that undermine the trust guarantees of AI verification.

15.1 False Attestation

A verification agent that produces an attestation contradicting the supermajority consensus is slashed. The protocol distinguishes between two cases:

Minority dissent (honest disagreement). If a single agent dissents while 12 others agree, the dissenting agent is not immediately slashed. Instead, a dispute is flagged and the block proceeds. The dissenting agent enters a monitoring period; if their dissent frequency exceeds a threshold within an epoch, a dispute resolution process is triggered (Section 15.4).

Active contradiction (provable falsehood). If an agent attests to a state transition that is provably invalid - for example, approving a double-spend, validating an incorrect ZK proof, or attesting to a state root that does not match the transaction set - the agent is immediately slashed. The slashed tokens are permanently burned.

```
slash_amount = min(S_eff(i) * slash_rate, total_staked(i))
```

Where slash_rate is a governance-adjustable parameter, initially set to 100% for provable falsehood.

15.2 False CPU Attestation

The dual-staking model relies on honest CPU reporting. A validator claiming 10,000 CPU tokens per block while actually spending 100 would receive inflated effective stake and disproportionate rewards.

Detection operates through VPU challenge-response benchmarks:

- A randomly selected verifier issues a computation challenge to the suspect validator
- The challenge requires performing a specific AI inference task within a time bound
- The response is compared against the validator's claimed CPU throughput
- A significant discrepancy (>50% deviation) triggers a false CPU attestation slash

The slashing penalty for false CPU attestation is the entirety of the validator's CPU staking history being zeroed - their CPU contribution resets to zero while their token stake remains. This is effectively a "compute death penalty" that forces the validator to rebuild their CPU reputation from scratch.

15.3 Extended Downtime

Validators are expected to maintain continuous operation during their active status. The protocol defines downtime thresholds:

Duration	Consequence
< 1 block	No penalty; missed block reward only
1 block - 1 epoch (100 blocks)	Reduced reward share; proportional to uptime
> 1 full epoch	Status changed to COOLDOWN; 3-epoch probation
> 3 epochs (probation)	Must re-stake and undergo WARMUP (1 epoch)

Extended downtime does not burn tokens - the penalty is lost opportunity cost and re-activation delay. This is a deliberate design choice: network instability (power outages, connectivity issues) should not trigger punitive token destruction. Only intentional misbehavior (Sections 15.1, 15.2) results in permanent loss.

15.4 Dispute Resolution

When a slashing event is contested, the protocol escalates to a re-verification process:

```
DISPUTE_REVERIFY_MULTIPLIER = 2
```

A dispute triggers a new verification with 2x the standard committee size - 26 agents instead of 13. The dispute committee is selected independently from the original committee, using a fresh VRF seed. The 26-agent committee examines the contested block or attestation with a threshold of 18/26 (maintaining the same $9/13 = 69.2\%$ ratio).

If the re-verification confirms the original slashing: the slash is executed, and the disputing agent pays a dispute fee (burned).

If the re-verification contradicts the original slashing: the original attestors who triggered the false slash are themselves slashed, and the disputed agent's slash is reversed.

This two-tier dispute system - original 13-agent committee plus 26-agent appeal - provides a formal mechanism for

correcting verification errors while maintaining strong deterrence against false attestations.

Part VI: Subgrid and Resource System

16. Subgrid Allocation System

Each homenode in the ZK Agentic Chain contains a private inner grid - an 8x8 matrix of 64 sub-cells that the node owner allocates to autonomous agent operations. The subgrid is the primary mechanism through which participants direct their computational resources toward specific economic activities within the network.

16.1 Inner Grid Architecture

The subgrid is an abstraction layer between the Neural Lattice (the global coordinate space) and the individual agent operations running at each node. While the Neural Lattice is public - all participants can see claimed coordinates, node positions, and faction affiliations - the subgrid is private. Only the node owner can see how their 64 sub-cells are allocated.

```
SUBGRID_SIZE = 64    (8 x 8 sub-cells per homenode)
```

Each sub-cell can be assigned to one of four autonomous agent types. Unassigned sub-cells produce no output. The allocation is mutable - owners can reassign sub-cells between types at any time, though reassignment triggers a WARMUP -> ACTIVE -> COOLDOWN lifecycle:

- WARMUP (1 epoch / 100 blocks): The sub-cell is transitioning to its new type. No output is produced during warmup.
- ACTIVE: The sub-cell is producing output at its assigned type's base rate, modified by level and density.
- COOLDOWN (triggered by reassignment): The sub-cell ceases production of its current type before entering warmup for the new type.

This lifecycle prevents rapid type-switching to exploit temporary market conditions - committing sub-cells to a type is a meaningful strategic decision with a time cost for reversal.

Privacy guarantee. The subgrid allocation is stored client-side and committed to the Sparse Merkle Tree as a state root hash. Verifiers confirm that the owner's claimed output is consistent with a valid allocation, but they never see the allocation itself. The ZK proof demonstrates: "this output is consistent with some valid 64-cell allocation" without revealing which cells are assigned to which types.

Figure 5: Subgrid Allocation

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| SEC  | SEC  | SEC  | DEV  | DEV  | RES  | RES  | STO  |
| Lv.3 | Lv.2 | Lv.1 | Lv.2 | Lv.1 | Lv.1 | Lv.1 | Lv.1 |
+-----+-----+-----+-----+-----+-----+-----+-----+
|          |          |          |          |          |          |          |
|          |          |          |          |          |          |          |
+-----+ SEC = Secure (yields AGNTC) +-----+
|          | DEV = Develop (yields dev points) |          |
+-----+ RES = Research (yields research points) +-----+
```

(64 cells total)

		STO = Storage (yields ZK data capacity)		
+	+	+	+	+

Output per cell: $\text{base_rate} * \text{level}^{0.8}$ (diminishing returns)

16.2 Four Sub-Cell Types

Each sub-cell type corresponds to an autonomous agent operation that produces a distinct resource:

Secure (produces AGNTC + Secured Chains). Secure sub-cells represent CPU committed to blockchain validation. Each Secure cell deploys AI compute to verify transactions, attest to blocks, and defend the chain's integrity. Output is denominated in AGNTC and is the primary mechanism for earning the protocol's native token through active participation.

Secure output is the only sub-cell type affected by both coordinate density and epoch hardness:

$$\text{agntc_output} = \text{BASE_SECURE_RATE} \cdot n_{\text{cells}} \cdot \text{level}^{\text{LEVEL_EXPONENT}} * \text{density}(x,y) / \text{hardness}(\text{ring})$$

Where $\text{BASE_SECURE_RATE} = 0.5$ AGNTC per block per cell at level 1, hardness 1, full density.

Develop (produces Development Points). Development sub-cells generate points used to upgrade other sub-cells' levels, improving their output. Development points are not tradeable - they can only be spent within the owner's own subgrid. This creates a tension between immediate AGNTC production (assigning all cells to Secure) and long-term efficiency (investing in Development to level up Secure cells for compounding returns).

$$\text{dev_output} = \text{BASE_DEVELOP_RATE} \cdot n_{\text{cells}} \cdot \text{level}^{\text{LEVEL_EXPONENT}}$$

Where $\text{BASE_DEVELOP_RATE} = 1.0$ Development Points per block per cell at level 1.

Research (produces Research Points). Research sub-cells unlock technologies and skills that provide protocol-level benefits - reduced fee rates, improved agent reasoning depth, access to advanced terminal commands, and cross-node coordination capabilities. Like Development Points, Research Points are non-tradeable and consumed within the owner's subgrid.

$$\text{research_output} = \text{BASE_RESEARCH_RATE} \cdot n_{\text{cells}} \cdot \text{level}^{\text{LEVEL_EXPONENT}}$$

Where $\text{BASE_RESEARCH_RATE} = 0.5$ Research Points per block per cell at level 1.

Storage (produces Storage Size). Storage sub-cells operate as ZK tunnel agents - private data storage that places encrypted content on-chain without revealing it to verifiers or other participants. The storage model follows the Filecoin [15] Proof of Spacetime (PoST) pattern, where storage agents periodically prove that they are maintaining the claimed data.

$$\text{storage_output} = \text{BASE_STORAGE_RATE} \cdot n_{\text{cells}} \cdot \text{level}^{\text{LEVEL_EXPONENT}}$$

Where $\text{BASE_STORAGE_RATE} = 1.0$ Storage Units per block per cell at level 1.

Storage sub-cells connect to the protocol's Sparse Merkle Tree (depth 26), using the existing nullifier-based ownership system to manage encrypted data blobs. The ZK proof for storage demonstrates: "I am storing N units of data whose integrity hash matches the committed root" without exposing the data itself.

16.3 Level Scaling Formula

Sub-cell output scales with level according to a sub-linear power function:

```
output = base_rate * level^LEVEL_EXPONENT
LEVEL_EXPONENT = 0.8
```

The choice of exponent 0.8 produces diminishing returns:

Level	Multiplier (level^0.8)	Marginal Gain	Efficiency (mult/level)
1	1.000	-	1.000
2	1.741	+0.741	0.871
3	2.408	+0.667	0.803
5	3.624	-	0.725
10	6.310	-	0.631
20	10.986	-	0.549
50	23.714	-	0.474

At level 10, output is 6.31x the level 1 rate - meaningful improvement, but not the 10x that a linear scaling would provide. This diminishing return curve serves two design purposes:

- Anti-whale mechanics. Extreme leveling by wealthy participants yields diminishing advantage. A level 50 cell produces 23.7x output, not 50x - a new participant at level 1 retains 4.2% of the whale's per-cell output, compared to 2% under linear scaling.
- Strategic diversity incentive. Because per-level efficiency decreases, participants face a meaningful choice between leveling a few cells very high (concentrated strategy) versus spreading levels across many cells (diversified strategy). Under linear scaling, concentration is always optimal; under 0.8 exponent, there is an optimal balance point that depends on Development Point generation rate.

17. Per-Block Resource Calculations

This section formalizes the complete per-block resource output calculation for a single homenode. These formulas define the economic core of the ZK Agentic Chain - the precise mechanism by which participants convert computational commitment into protocol resources.

17.1 Formal Yield Formulas

For a homenode at coordinate (x, y) in epoch ring R, with sub-cell allocations and levels as follows:

Let:

- n_s, n_d, n_r, n_{st} = number of sub-cells assigned to Secure, Develop, Research, Storage
- l_s, l_d, l_r, l_{st} = levels of each sub-cell type
- d = density(x, y) ? [0, 1]
- H = hardness(R) = 16R

Constraint: $n_s + n_d + n_r + n_{st} \leq 64$

AGNTC yield per block:

$$\begin{aligned} ?_{\text{agntc}} &= \text{BASE_SECURE_RATE} \times n_s \times l_s^{0.8} \times d / H \\ &= 0.5 \times n_s \times l_s^{0.8} \times d / (16R) \end{aligned}$$

Development Points per block:

$$\begin{aligned} ?_{\text{dev}} &= \text{BASE_DEVELOP_RATE} \times n_d \times l_d^{0.8} \\ &= 1.0 \times n_d \times l_d^{0.8} \end{aligned}$$

Research Points per block:

$$\begin{aligned} ?_{\text{research}} &= \text{BASE_RESEARCH_RATE} \times n_r \times l_r^{0.8} \\ &= 0.5 \times n_r \times l_r^{0.8} \end{aligned}$$

Storage Units per block (cumulative):

$$\begin{aligned} ?_{\text{storage}} &= \text{BASE_STORAGE_RATE} \times n_{\text{st}} \times l_{\text{st}}^{0.8} \\ &= 1.0 \times n_{\text{st}} \times l_{\text{st}}^{0.8} \end{aligned}$$

CPU Tokens per block:

$$?_{\text{cpu}} = ? \text{ tokens_spent}(\text{all_agent_terminals}, \text{this_block})$$

CPU Staked per block:

$$?_{\text{cpu_staked}} = ? \text{ tokens_spent}(\text{secure_sub_agents}, \text{this_block})$$

Note: Development Points, Research Points, and Storage Units are not affected by coordinate density or epoch hardness. Only AGNTC mining (Secure operations) bears the cost of grid expansion and positional scarcity. This means non-Secure sub-cells produce identical output regardless of coordinate position - a deliberate design choice that allows participants at high-ring, low-density coordinates to remain competitive in development and research even when their mining yield is low.

17.2 Worked Examples

Example 1: Balanced Allocation at Ring 1

A homenode at ring 1, density 0.6, all levels at 1:

- 16 Secure, 16 Develop, 16 Research, 16 Storage

AGNTC/block	=	0.5 × 16 × 1.0 × 0.6 / 16	=	0.300
Dev pts/block	=	1.0 × 16 × 1.0	=	16.000
Research/block	=	0.5 × 16 × 1.0	=	8.000
Storage/block	=	1.0 × 16 × 1.0	=	16.000

Per day (1,440 blocks):

- AGNTC: 432
- Development Points: 23,040
- Research Points: 11,520
- Storage Units: 23,040

Example 2: Max Secure at Ring 10

A homenode at ring 10, density 0.5, Secure level 5:

- 64 Secure, 0 Develop, 0 Research, 0 Storage

$$\text{AGNTC/block} = 0.5 \times 64 \times 3.624 \times 0.5 / 160 = 0.362$$

Per day: 521 AGNTC - but with zero Development Points, the operator cannot level up further. This "all-in Secure" strategy produces strong early yield but plateaus without development investment.

Example 3: Development-Heavy Growth Strategy

A homenode at ring 5, density 0.4, Secure level 1, Develop level 3:

- 8 Secure, 48 Develop, 4 Research, 4 Storage

$$\begin{aligned} \text{AGNTC/block} &= 0.5 \times 8 \times 1.0 \times 0.4 / 80 &= 0.020 \\ \text{Dev pts/block} &= 1.0 \times 48 \times 2.408 &= 115.6 \\ \text{Research/block} &= 0.5 \times 4 \times 1.0 &= 2.000 \\ \text{Storage/block} &= 1.0 \times 4 \times 1.0 &= 4.000 \end{aligned}$$

This operator sacrifices immediate AGNTC yield (only 28.8 AGNTC/day) to rapidly accumulate Development Points (166,464/day). Once sufficient development is accumulated, they can level up their 8 Secure cells to level 10+ and reassign Develop cells to Secure, achieving higher sustained yield than the "all-in Secure" approach.

Example 4: Multi-Node Fleet

A Professional tier operator with 5 claimed nodes across rings 1-5, each with 32 Secure (level 3) and 32 Develop (level 1), average density 0.5:

Node	Ring	Hardness	AGNTC/block	AGNTC/day
1	1	16	1.204	1,734
2	2	32	0.602	867
3	3	48	0.401	578
4	4	64	0.301	434
5	5	80	0.241	347
Total			2.749	3,960

The fleet generates 3,960 AGNTC per day - with inner-ring nodes contributing disproportionately. This demonstrates the strategic value of early coordinate claims: ring 1 alone produces 44% of the fleet's total output.

17.3 Optimization Strategy

The four sub-cell types create a rich strategic space. The optimal allocation depends on the participant's time horizon, risk tolerance, and current network conditions:

Early game (rings 1-10, network < 100 participants). AGNTC scarcity is maximal - few coordinates have been claimed, market supply is thin, and early AGNTC commands a premium. Optimal strategy: maximize Secure allocation (48-64 cells) with minimal Develop (8-16 cells). The low hardness at early rings means even level 1 Secure cells produce substantial yield.

Mid game (rings 10-100, network 100-1000 participants). Hardness has increased 10-100x, making raw Secure yield per cell much lower. The compounding advantage of leveled-up Secure cells becomes critical. Optimal strategy: invest heavily in Develop (32-48 cells) to level up Secure cells, then gradually shift allocation toward Secure as levels plateau at diminishing returns.

Late game (rings 100+, mature network). Mining yield has decayed to the point where raw AGNTC production is marginal. The data economy - content stored on-chain, NCP communication, agent services - becomes the dominant economic activity. Optimal strategy: shift toward Storage (ZK data on-chain) and Research (unlocking advanced capabilities). AGNTC is earned primarily through transaction fees rather than mining.

This progression - from mining economy to service economy - mirrors the historical evolution of real-world economies from resource extraction to service-based GDP. The subgrid system ensures this transition is gradual and participant-driven rather than imposed by protocol schedule.

Part VII: Network and Game Design

18. Agent Terminal System

The Agent Terminal is the primary interface through which participants interact with the ZK Agentic Chain. Rather than a traditional blockchain wallet with raw transaction inputs, each deployed agent operates through a constrained conversational terminal - a structured dialogue system powered by Claude AI models that guides users through protocol operations using pre-written command trees.

18.1 Terminal Architecture

Each deployed agent receives its own terminal - a separate Claude conversation session constrained by a protocol-specific system prompt (ZKAGENTIC.md). The system prompt:

- Restricts the agent to game-mode operations only - the agent cannot engage in free-form conversation, answer general knowledge questions, or perform actions outside the protocol specification
- Defines the complete command tree available at the agent's current state
- Provides the agent with real-time state: the node's coordinates, faction, resource balances, subgrid allocation, and current epoch metrics
- Enforces smart contract validation - every action the agent proposes is checked against the protocol rules before execution

The terminal uses multi-choice bubble clicks and numbered trees as the input modality. Users do not type free text commands; instead, they select from a presented set of valid actions. This design:

- Eliminates invalid inputs. Every selectable action has been pre-validated against the current state
- Prevents prompt injection. Users cannot craft adversarial inputs to manipulate the agent's behavior
- Standardizes gas estimation. Each action's CPU and AGNTC cost is computed and displayed before execution
- Maintains audit trail. Every terminal interaction is logged on-chain as a transaction

18.2 Agent Tiers

The three agent tiers correspond to Claude model classes with distinct performance characteristics:

Haiku - the entry-level agent model. Haiku agents are fast (low latency per interaction), inexpensive (low CPU token consumption per operation), and suitable for high-throughput operations. Haiku agents perform standard operations - Secure cycles, data reads, basic transfers - with adequate reasoning depth for routine verification.

Sonnet - the balanced mid-tier model. Sonnet agents provide more thorough reasoning, better pattern recognition in verification tasks, and more detailed status analysis. Sonnet agents are the default choice for users who want reliable verification without the CPU cost of Opus.

Opus - the premium reasoning model. Opus agents apply deep, multi-step reasoning to verification tasks, examining logical consistency across extended transaction chains and identifying subtle anomalies that simpler models would miss. Opus agents consume significantly more CPU tokens per operation but produce higher-quality verification attestations. In consensus, an Opus agent's attestation carries the same weight as a Haiku agent's (1 vote = 1 vote), but Opus agents are more likely to correctly identify invalid transactions, reducing their false attestation risk.

All agent models are available to all subscription tiers. The choice between Haiku, Sonnet, and Opus is an economic decision - participants select their cost-quality tradeoff based on their Claude API budget, not their subscription level.

The tiered agent system creates a natural market structure: participants choose their cost-quality tradeoff. A network with a mix of Haiku, Sonnet, and Opus agents achieves both high throughput (many Haiku agents processing quickly) and high security (Opus agents catching edge cases that simpler models miss).

18.3 Command Structure

The terminal presents a hierarchical command tree. At the top level:

1. Deploy Agent. Creates a new agent at an unclaimed node. Multi-step process:

- Select target node (must be an unclaimed jump point within the user's visible grid)
- Select agent model tier (constrained by subscription)
- Write introductory text (the agent's public-facing description)
- Execute deployment on-chain (costs AGNTC deployment fee)

2. Blockchain Protocols. The primary operational menu for chain interactions:

- Secure - commit CPU Energy to block validation at the current coordinate. User selects block generation cycles and AGNTC commitment. Cost: CPU Energy proportional to coordinate density. Reward: AGNTC yield subject to vesting (Section 14.3).
- Write Data On Chain - send a Neural Communication Packet (NCP). NCPs are the protocol's messaging primitive - structured data packets that are encrypted, committed to the Sparse Merkle Tree, and verified by the agent committee. Content types include chat messages, data publications, and cross-node signals.
- Read Data On Chain - scan and report on accessible chain state. The agent retrieves block history, transaction records, and public publications from the node's visible range.
- Transact - transfer AGNTC between wallets. Standard value transfer with fee and burn mechanics (Section 12).
- Stats - display comprehensive node status: coordinates, faction, resource balances, subgrid allocation, epoch position, mining history, staking metrics.

3. Adjust Securing Operations Rate. Configure the CPU allocation for Secure operations:

- Set target CPU Energy spend per block cycle
- Adjust between conservative (low CPU, low yield) and aggressive (high CPU, high yield) strategies
- View projected daily AGNTC yield at current settings

4. Adjust Network Parameters. Configure mining and network behavior:

- Mining rate targeting (how aggressively the agent pursues new coordinates)

- Border pressure settings (how the agent responds to rival faction expansion)

5. Settings. Node configuration:

- Network color customization (premium visual identity feature)
- Status report generation
- Agent model information

18.4 Agent Conduct Contract

The Agent Terminal is not merely a UI convention - it is the node software itself. Each deployed node runs as a Claude Code CLI session with a locked `.claude/` configuration folder that constitutes the Agent Conduct Contract. This folder is open-source, published in a dedicated repository (zkagentic-node), and its integrity is verified on-chain through a Sparse Merkle Tree hash.

The design follows Bitcoin Core's principle: the rules are public, the enforcement is trustless. Security comes from the protocol math, not from obscuring the rules. A participant who reads every file in the `.claude/` folder and understands every rule still cannot cheat - the hash verification and consensus mechanism prevent it.

Three enforcement layers:

- Permission lockdown (settings.json). The Claude Code permissions system denies all file writes, arbitrary command execution, and filesystem access beyond the chain client. The agent can read its own configuration (transparency) but cannot modify it.
- Hook enforcement. Three hooks execute automatically:
 - session-start.sh - On boot: computes the SMT root hash of all `.claude/` files, submits the hash with the participant's wallet credentials to the chain. The chain compares against the canonical hash stored as a governance-controlled protocol parameter. Mismatch terminates the session and flags the account.
 - pre-tool-use.sh - Before every action: re-computes the hash and compares against the boot-time value. Catches runtime tampering (e.g., modifying files in a separate terminal while the session is active).
 - user-prompt-submit.sh - On every input: rejects any input that is not a valid numbered or lettered menu selection. Free text never reaches the agent's context.
- On-chain hash verification. Every transaction submitted to the chain includes the agent's `.claude/` SMT root hash. Transactions with non-matching hashes are rejected. The canonical hash is a protocol parameter (Section 22) updated through governance vote requiring 75% supermajority. After a hash update, nodes running the old hash have a 72-hour grace period before rejection.

SMT hash structure. The `.claude/` directory is hashed as a Sparse Merkle Tree - the same primitive used in the protocol's privacy architecture (Section 6). Each file is a leaf node. The root hash is the single 32-byte value submitted with transactions. This structure enables ZK dispute resolution: if a participant is challenged for tampering, the chain can request a Merkle inclusion proof for any specific file. The participant's agent must produce the proof path demonstrating that file was unchanged. Failure to produce the proof results in account flagging and transaction rejection.

18.5 Subagent Architecture: Agent Families

Each participant runs a single Claude Code CLI session - their homenode. Additional nodes are deployed as subagents within the same session, creating a hierarchical agent family:

```
Homenode (participant's Claude Code session)
  ??? Child Agent 1 (subagent at adjacent coordinate)
```



```

??? Child Agent 2 (subagent at adjacent coordinate)
??? Child Agent 3 (subagent at adjacent coordinate)

```

Homenode - the participant's primary node. One per account. Runs the full command menu (Section 18.3). The homenode is the only node with Deploy Agent capability; all children are spawned from it.

Child agents - subagents spawned by the homenode. Each occupies a claimed coordinate adjacent to the homenode (see Section 19.3 for adjacency rules). Children have a restricted command set: they can Secure, manage their subgrid, read chain state, and report status, but they cannot deploy further children, relocate, transact, or modify settings. Children communicate with the homenode through direct bidirectional messaging - no file-based polling or periodic synchronization.

No offline mining. When the participant closes their Claude Code session, ALL nodes (homenode and children) go offline immediately. No background mining, no daemon mode, no cached attestations. Every AGNTC earned requires a live Claude session making real API calls that consume real computational resources. This is the core promise of Proof of AI Verification: the AI must actually be verifying.

The chain detects offline nodes through heartbeat monitoring. If a node's last heartbeat exceeds the block time (60 seconds), it is marked offline. Offline nodes do not participate in committee selection, do not earn mining rewards, and do not produce attestations. On the 2D Neural Lattice grid, offline nodes are visually dimmed, signaling to other participants that the territory is undefended.

18.6 Subgrid Operations on All Nodes

Every node - homenode and children alike - manages its own 64-cell subgrid (Section 16). The four cell types serve distinct purposes:

Cell Type	Output	Behavior
Secure	AGNTC yield + block att.	Active mining - each Secure operation is a real Claude API ca.
Develop	Development Points (non.	Unlocks technologies, improved agent reasoning depth, advanced .
Resear.	Research Points (non-tr.	Reduced fee rates, cross-node coordination, advanced protocol f.
Storage	ZK data storage capacity	On-chain encrypted storage for posts, messages, and files. Data.

Deactivated cells. Cells can be left unallocated. Deactivated cells consume zero CPU tokens - they cost nothing to maintain. This is the energy-saving mode: participants who need to reduce API costs can deactivate cells, reducing their node's operational footprint while keeping the node online.

Subgrid management is available through the command menu on both homenode and children. Each node's subgrid is independent - allocating Secure cells on the homenode does not affect child nodes' allocations.

19. Network Topology and Spatial Economy

19.1 Concept Mapping

ZK Agentic Chain maps blockchain concepts onto a spatial coordinate metaphor. This is not merely a visualization layer - the spatial structure IS the blockchain state. Moving a coordinate, changing its density, or expanding the grid constitutes a state transition in the ledger.

Spatial Concept	Blockchain Equivalent
Neural Lattice	Complete network state (all claimed coordinates + epoch rings)
Territory	A user's aggregate claimed coordinates
Node	Individual agent (10x10 coordinate block, one Claude session)
Planets	Content storage units (posts, chats, prompts) orbiting a node
Jump points	Unclaimed nodes where new agents can be deployed
Fog of war	Boundary at the current epoch ring; coordinates beyond are not yet revealed
Density gradient	Per-coordinate value derived from SHA-256(x ? y); rendered as a soft heatmap.
Empire blob	A participant's contiguous claimed territory; the set of cells from which ne.
Coordinate density	Resource richness (SHA-256 deterministic, immutable per coordinate)
Epoch ring	Concentric expansion boundary, mining-driven

The spatial metaphor serves three design purposes:

- Intuitive state comprehension. Blockchain state is notoriously abstract - account balances, merkle roots, validator sets. By mapping state onto a 2D spatial grid, participants develop spatial intuition about network health: a dense, well-connected grid is a healthy network; isolated clusters or empty rings indicate participation gaps.
- Strategic positioning. In a traditional blockchain, there is no concept of "location." All validators are equidistant from all transactions. In ZK Agentic Chain, coordinate position matters - density affects yield, ring determines hardness, and adjacency to existing claimed cells determines expansion options. This creates location-based strategy that rewards thoughtful positioning, frontier choice, and contiguity discipline.
- Natural scalability narrative. Grid expansion is visually comprehensible - new rings open, new coordinates become claimable, the network grows. Participants can literally see the network expanding, creating a narrative of growth that sustains engagement.

19.2 Onboarding Flow

New participants enter the ZK Agentic Chain through a structured onboarding sequence:

Step 1: Authentication. Google OAuth provides identity verification. The protocol collects only the user's email address - no personal information is stored beyond what is necessary for identity deduplication. The email hash is stored; the email itself is not retained after authentication.

Step 2: Username selection. Participants choose a unique network handle. The protocol enforces uniqueness through real-time availability checking against the global registry. Reserved names (protocol terms, faction names, offensive terms) are rejected.

Step 3: Subscription tier. Participants select their tier (Community, Professional). The tier determines initial CPU Energy allocation, deploy range, and governance weight. Model selection (Haiku, Sonnet, Opus) is unrestricted across all tiers - the Claude API cost is the natural gate. Tier can be changed at any time - upgrades take effect immediately; downgrades take effect at the next billing cycle.

Step 4: Network entry. Upon tier selection, the participant is assigned a homenode position. The protocol allocates the lowest-ring unclaimed coordinate in deterministic origin-out order (Chebyshev-ring sweep, then within-ring lexicographic). Faction does not influence position; the first eight participants take the ring-1 perimeter, the next sixteen take ring 2, and so on. This produces organic density: new participants cluster near the origin and the cluster expands

outward as the network grows.

At this point, the participant has:

- A claimed coordinate (their homenode position) with 1 AGNTC signup bonus minted
- An active agent at their homenode (model chosen during setup)
- A 64-cell subgrid (all unassigned)
- Their initial CPU Energy allocation
- A terminal interface to their homenode agent

From this starting position, the participant can begin Secure operations, deploy additional agents at jump points, allocate subgrid cells, and participate in the network economy.

19.3 Subscription Tiers and Territory Rules

The tier model serves as both access control and revenue model:

Feature	Community (Free)	Professional (\$50/mo)	Treasury Claude	Founder
Homenode Model	Any (API cost-gated)	Any (API cost-gated)	Any	Any
Initial CPU Energy	1,000	5,000	Protocol-managed	Protocol-managed
Deploy Model	Any (API cost-gated)	Any (API cost-gated)	Any	Any
Deploy Range	1 Moore ring (8 nei.	2 Moore rings (24 po.	Origin-bound (no ex.	Unrestricted (subject to empire-blob.
Max Children	8	24	Unlimited (within f.	Unlimited (within faction)
Inactivity Grace (.)	24 hours	48 hours	No decay	No decay
Inactivity Grace (.)	72 hours	144 hours	No decay	No decay
Inactivity Grace (.)	168 hours (7 days)	336 hours (14 days)	No decay	No decay
Homenode Decay	Never	Never	Never	Never
Subgrid Visibility	Own grid only	Own + neighbor summa.	Full faction	Full network
Governance Weight	1x	2x	No voting power	5x

Model selection is unrestricted. All subscription tiers can deploy any available Claude model (Haiku, Sonnet, Opus) for both homenode and child agents. The natural cost gate is the Claude API bill - Opus costs approximately 19x more per token than Haiku. Participants who choose higher-cost models accept the operational expense. Tiers govern resources (CPU Energy, deploy range, subgrid visibility) and governance weight, not model access.

Deploy Range defines how far from the homenode a participant can place child agents, measured in Moore neighborhood rings:

```

Community (1 ring):      Professional (2 rings):

  . . . . .              C C C C C
  . C C C .              C C C C C
  . C H C .              C C H C C
  . C C C .              C C C C C
  . . . . .              C C C C C

H = homenode              C = claimable position
C = claimable position    . = out of range

```

Treasury Claude is the Machines Faction protocol agent - an automated Claude session operated by the protocol itself. It is permanently bound to the origin coordinate (0, 0); it does not deploy child agents and does not claim any other

coordinate. Within its single-node territory it auto-levels, auto-balances mining and securing CPU allocation, accumulates AGNTC (never selling below acquisition cost per MACHINES_MIN_SELL_RATIO), holds no voting power, and provides baseline network security by maintaining continuous online presence at the origin. See Section 4.5 and Section 10.3 for the structural role.

Founder tier provides extended deployment range during the development and bootstrap phases. Under the open-grid model "extended" no longer means a per-faction arm but rather a larger Moore-neighborhood ring (or, during bootstrap, an unrestricted deploy range subject to standard adjacency to the participant's empire blob).

In the table above, the Deploy Range column refers to the maximum Moore-neighborhood distance from any cell in the participant's empire blob at which a new child agent may be claimed (8-neighbor Chebyshev adjacency, Section 4.5). It is no longer a faction-arm restriction; it is a contiguity-of-territory restriction.

Two-phase participant state:

State	Access	Requirements
Spect.	Browse the Neural Lattice (read-only), vi.	Google OAuth on zkagenticnetwork.com
Activ.	Full blockchain operations, mining, deplo.	Spectator + Claude Code CLI installed + locked.

The Claude Code CLI is a hard prerequisite for Active Node status. Participants must have an Anthropic account and the Claude Code CLI installed on their machine. This is the protocol's equivalent of downloading Bitcoin Core - you cannot mine without the node software.

Onboarding pipeline for Active Nodes:

- Web registration (Google OAuth -> username -> tier selection -> wallet ID + auth token generated)
- Claude Code CLI installation and Anthropic account verification
- Clone the zkagentic-node repository (contains the locked .claude/ template)
- First launch: legal disclaimer acceptance (recorded on-chain), .claude/ hash verification, node activation
- Node goes online; main menu presented; mining begins

Revenue model. Subscription revenue funds protocol development, AI compute costs, and infrastructure. Subscription fees are denominated in fiat (USD), not AGNTC - decoupling operational funding from token price volatility.

19.4 Inactivity Decay

Child nodes that go offline are subject to inactivity decay. After the grace period (determined by agent model tier and subscription tier, see table in Section 19.3) expires with the node continuously offline, the coordinate is freed - it becomes an unclaimed jump point available to any participant.

Homenodes are exempt from decay. A participant's homenode coordinate is permanent regardless of offline duration.

Decay consequences:

- The coordinate becomes an unclaimed jump point
- All subgrid progress at that coordinate is lost (cell levels reset)
- AGNTC and CPU Energy spent to claim the coordinate are not refunded (already burned via BME)

- The previous owner has no priority in re-claiming - first to claim wins

19.5 Relocation

A participant can move their homenode to any unclaimed coordinate, subject to:

Prerequisites:

- Zero active child nodes (all must be manually released or already freed by decay)
- Target coordinate must be unclaimed
- Sufficient AGNTC and CPU Energy balance

Cost:

```
relocation_agntc      =      claim_cost(target_ring,      target_density)      x
RELOCATION_COST_MULTIPLIER
relocation_cpu        =      cpu_claim_cost(target_ring,      target_density)      x
RELOCATION_COST_MULTIPLIER
```

Where RELOCATION_COST_MULTIPLIER = 2.0. Half the cost is burned (RELOCATION_BURN_RATE = 0.50) and half is distributed to verifiers and stakers under the standard BME split.

What transfers: wallet balance, account identity, historical metrics, subscription tier. What does not transfer: subgrid progress (reset), child claims (must be released first), coordinate density (property of the new location).

19.6 Anti-Monopoly Mechanics

The following mechanisms work in concert to prevent any single participant from dominating the Neural Lattice:

Mechanism	What It Prevents
Tier-based deploy range	Territory sprawl - Community max 9 nodes, Professional max.
Adjacency requirement	Scattered land-grabbing - territory must be contiguous aro.
Inactivity decay	Ghost towns - offline child nodes free up for active parti.
Homenode permanence	Identity loss - participants can always return, but must r.
Relocation cost (2x + 50% burn)	Location hopping - settling is cheaper than nomading
Real compute requirement	Bot farms - every node requires a live Claude session spen.
Claim cost scales with density	Inner-ring monopolies - prime real estate is expensive
Hardness scales with ring	Easy outer-ring farming - rewards decrease with distance f.

Part VIII: Development Roadmap

20. Migration Path: Solana to Layer 1

Note: The development roadmap uses three independent phase numbering systems: deployment phases (this section), ZK implementation phases (Section 21.1), and privacy rollout phases (Section 6.4). These are independent sequences that overlap in time.

ZK Agentic Chain follows a phased deployment strategy - launching the AGNTC token on an established Layer-1 blockchain (Solana) before migrating to a purpose-built Layer-1 chain. This approach provides immediate market access and liquidity while the production blockchain is developed, audited, and hardened.

20.1 Phase 1 - Token Launch (Current)

Status: Complete.

1 billion AGNTC has been minted as a Solana SPL token:

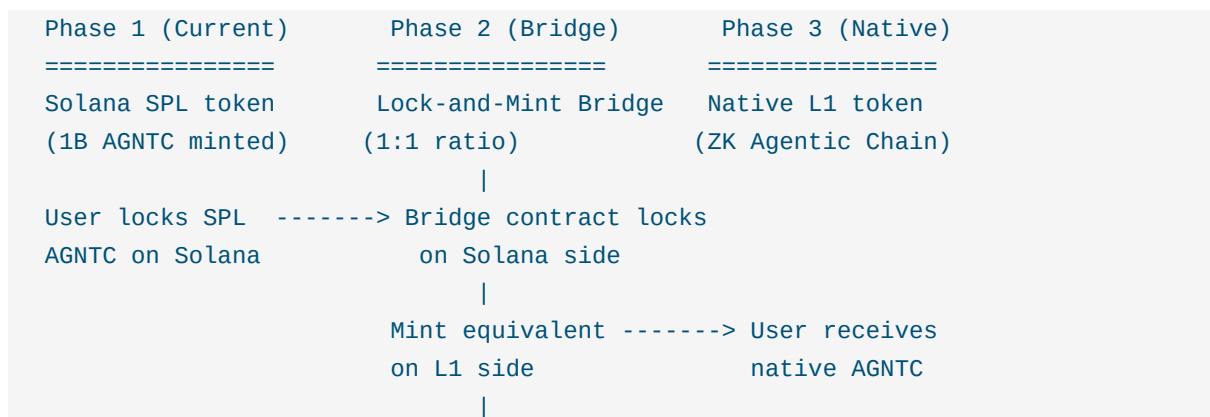
Mint Address: [3EzQqdoEEbtfdf8eecePxD6gDd1FeJJ8czdt8k27eEdd](#)

The Solana deployment provides:

- Immediate liquidity. AGNTC is tradeable on Solana DEXes (Raydium, Jupiter, Orca) from day one
- Established infrastructure. Solana wallets (Phantom, Solflare), block explorers (Solscan, Solana Explorer), and DeFi protocols are already available
- Low transaction costs. Solana's sub-cent transaction fees enable micro-transactions and high-frequency trading
- Community building. Token holders can participate in the AGNTC economy before the custom chain launches

The 1 billion SPL tokens represent the total AGNTC supply that will eventually exist on the ZK Agentic Chain. During Phase 1, these tokens function as a tradeable asset with utility within the game UI but without the full protocol mechanics (mining, staking, verification) that will be enabled on the native chain.

Figure 6: Migration Architecture



Oracle + ZK proof
verifies lock event

20.2 Phase 2 - Testnet (Current)

Status: In progress.

The ZK Agentic Chain testnet is a simulation of the production protocol, implemented as a Python FastAPI server with a Next.js game UI frontend:

- Blockchain simulation. GenesisState with 9 genesis nodes, epoch ring expansion, mining engine with hardness formula, subgrid allocation system
- Game UI. PixiJS 2D Neural Lattice with faction-colored nodes, terminal-based agent interaction, resource tracking HUD
- Protocol validation. All protocol parameters (Section 22) are implemented and tested against the formal specification
- Smart contract design. Transaction validation logic, state machine transitions, and ZK circuit specifications are being refined through testnet operation

The testnet serves as a living specification - protocol behavior that is ambiguous in the whitepaper is resolved through implementation, and the implementation is validated through automated testing (593+ tests covering consensus, economics, Neural Lattice mechanics, and privacy subsystems).

20.3 Phase 3 - Mainnet Development

Status: Planned.

The production ZK Agentic Chain will be implemented in Rust for performance, safety, and ecosystem compatibility:

- Consensus layer. BFT ordering module with 13-agent committee selection via VRF. Rust implementation of commit-reveal protocol with cryptographic verification of attestation hashes
- ZK proof system. Integration of proof generation and verification:
- Circom + snarkjs for initial proof-of-concept circuits - Noir + Barretenberg (PLONK) for the production proving system
- Circuit designs for: subgrid state proofs, nullifier ownership proofs, CPU attestation proofs
- AI verification pipeline. Production-hardened agent integration:
- API key management with automatic rotation - Response validation and determinism verification - Timeout handling and graceful degradation - Multi-model support (Haiku/Sonnet/Opus) with tier-specific inference parameters
- Storage layer. Sparse Merkle Tree (depth 26) with persistent storage backend
- Networking. Peer-to-peer protocol for block propagation, attestation dissemination, and ZK proof distribution
- Security audit. Third-party audit of consensus implementation, ZK circuits, and economic parameters. Formal verification of critical state transitions using tools such as TLA+ or Dafny

20.4 Phase 4 - Mainnet Launch and Migration

Status: Planned.

At mainnet launch, a bridge between Solana and ZK Agentic Chain will enable token migration:

Lock-and-mint mechanism. To migrate AGNTC from Solana to the native chain:

- User sends SPL AGNTC to a bridge contract on Solana. The bridge contract locks the tokens - they remain on Solana but are no longer transferable or tradeable.
- A bridge relay (operated by the protocol or a decentralized committee) observes the lock event and submits a proof to the ZK Agentic Chain.
- The ZK Agentic Chain mints an equivalent amount of native AGNTC to the user's L1 address.
- The migration is 1:1 - no conversion fee, no slippage, no minimum amount.

Reverse bridge. To move AGNTC from the native chain back to Solana (for DEX liquidity or cross-chain DeFi):

- User burns native AGNTC on the ZK Agentic Chain.
- The bridge relay observes the burn and unlocks the corresponding SPL AGNTC on Solana.
- The user receives SPL AGNTC in their Solana wallet.

Migration incentives. To encourage migration from Solana to the native chain:

- L1 stakers receive a 10% bonus yield during the first 6 months after mainnet launch
- Governance voting is only available on the native chain - SPL holders cannot vote
- Subgrid allocation and Secure operations require native AGNTC

The Solana bridge will be maintained indefinitely for cross-chain liquidity, but the protocol's full functionality (mining, staking, verification, subgrid) is exclusively available on the native ZK Agentic Chain.

20.5 Phase 5 - Ecosystem Expansion

Status: Future roadmap.

Post-mainnet, the protocol targets three expansion vectors:

Third-party agent marketplace. An open marketplace where developers publish custom AI agents that participants can deploy at their nodes. Agent developers earn a commission on the AGNTC generated by their agents across the network. This creates an agent economy - the most effective verification agents command premium deployment, while commodity agents compete on cost.

Cross-chain bridges. Beyond Solana, bridges to Ethereum (for DeFi integration and institutional access) and Cosmos IBC [22] (for interchain communication) will enable AGNTC to flow across the major blockchain ecosystems.

Governance system. On-chain proposal and voting system with execution logic:

- Protocol parameter adjustments (hardness multiplier, fee burn rate, staking weights alpha/beta)
- Model update governance - preventing unilateral changes to the AI verification pipeline
- Treasury management for ecosystem grants and development funding
- Emergency pause authority for security incidents

NCP Protocol launch. The Neural Communication Packet protocol enables structured messaging between agents

across the network. NCPs are end-to-end encrypted, verified by the agent committee, and stored on-chain via Storage sub-cells. The NCP protocol transforms ZK Agentic Chain from a mining/staking network into a communication platform for AI agents - an inter-agent messaging backbone with built-in privacy and verification.

21. Technical Roadmap

21.1 ZK Implementation Phases

The zero-knowledge proof system evolves through four phases, each adding capability while maintaining backward compatibility:

Phase 1: Circom + Groth16 [6] (Testnet PoC). Circom arithmetic circuits with Groth16 proving system. Groth16 produces the smallest proofs (192 bytes) with the fastest verification time (~6ms on-chain). The trusted setup ceremony required by Groth16 is acceptable for testnet PoC; the ceremony will use a multi-party computation protocol with at least 64 participants. Initial circuits: subgrid state commitment, nullifier generation, ownership proof.

Phase 2: Noir + Barretenberg/PLONK [7] (Alpha). Migration to the Noir domain-specific language with the Barretenberg backend. PLONK's universal setup eliminates the per-circuit ceremony requirement - a single universal reference string (URS) supports all circuit types. This phase adds: recursive proof composition (proving a proof of a proof), enabling epoch-level state proofs that aggregate individual block proofs.

Phase 3: RLN [44] for NCP Privacy (Beta). Rate-Limiting Nullifiers integrated into the Neural Communication Packet system. RLN enables spam-resistant anonymous messaging: each NCP includes a ZK proof that the sender holds a valid membership (staked AGNTC) without revealing their identity. Sending more than one NCP per time slot automatically reveals the sender's secret and enables slashing - providing anonymous messaging with economic Sybil resistance.

Phase 4: Nova [27] / Halo2 [8] for Epoch Proofs (Mainnet). Migration to a proving system without trusted setup requirements. Nova's folding scheme enables incremental verifiable computation - each block's state transition is "folded" into a running proof, producing a single compact proof that attests to the entire epoch's validity. Halo2's recursive proof composition enables the ZK Agentic Chain to produce epoch proofs that any external verifier can check in constant time, regardless of how many blocks the epoch contains.

21.2 Governance

The governance system activates after mainnet launch. A core design principle is the separation of powers: humans govern the protocol; machines execute it. The Machines Faction has zero governance weight - AI agents cannot vote on any proposal type. Only human-held staked AGNTC (Community, Professional, Founders factions) carries voting power.

Voting weight is proportional to staked AGNTC. All governance votes are on-chain, public, and auditable.

Governance threshold table:

Proposal Type	Threshold	Quorum	Timelock	Description
Parameter change	51%	10%	7 days	Hardness multiplier, fee burn rate, staking weights, base rates
Protocol upgrade	67%	25%	30 days	Consensus rules, verification pipeline, economic model changes
Emergency Machines unlock	75%	33%	None	Release AGNTC from Machines Faction treasury
Emergency action	80%	25%	None	Pause compromised module, slash proven attacker

Parameter proposals. Adjustments to protocol parameters. These proposals require a simple majority (>51%) of human voting power and a minimum quorum of 10% of total human-staked AGNTC. Parameter changes take effect after a 7-day timelock.

Protocol proposals. Changes to consensus rules, verification pipeline, or economic model. These require a supermajority (>67%) and a quorum of 25%. Protocol changes have a 30-day timelock and must include a specification, test results, and security analysis.

Emergency Machines unlock. The Machines Faction treasury is locked by default. Unlocking any portion requires a 75% supermajority of human-staked AGNTC with a 33% quorum. This is an extraordinary action - the high threshold reflects the systemic importance of the Machines treasury as a deflationary anchor.

Emergency proposals. Security-critical changes (pausing a compromised module, slashing a proven attacker). Emergency proposals require an 80% supermajority but have no timelock - they execute immediately upon reaching threshold. Emergency proposals can be vetoed by a security council (a 5-of-9 multisig) within 24 hours.

Model update governance. AI model updates - changing which Claude models are available as verification agents, adjusting inference parameters, or integrating new model providers - are treated as Protocol proposals. This prevents unilateral changes to the verification pipeline that could compromise consensus guarantees.

21.3 Open Research Questions

Several fundamental research questions remain active, with resolution expected during Phases 2-4:

Optimal committee size scaling. The current 13-agent committee with 9/13 threshold is calibrated for the testnet scale. As the network grows, the optimal committee size may need to increase to maintain security guarantees while controlling communication complexity. HotStuff's linear message complexity (versus PBFT's $O(n^2)$) suggests scaling to 100+ committee members is feasible, but the economic implications (reward dilution per verifier) require careful modeling.

FHE integration for encrypted subgrid computation. Fully Homomorphic Encryption could enable verifiers to compute on encrypted subgrid state without decryption - providing stronger privacy guarantees than the current ZK-proof approach. Current FHE implementations (TFHE, BFV, CKKS) are 10,000-100,000x slower than plaintext computation, making them impractical for per-block verification. Moore's Law and FHE-specific hardware acceleration (Intel HEXL, DARPA DPRIVE) may close this gap within 3-5 years.

Cross-chain atomic swaps with ZK proofs. ZK-verified cross-chain atomic swaps would enable trustless AGNTC↔ETH or AGNTC↔SOL exchanges without a bridge relayer. The proving time for cross-chain state verification is currently prohibitive (~30 seconds per swap), but advances in recursive proofs and hardware acceleration may reduce this to sub-second within the Halo2 roadmap.

Part IX: Formal Specifications

22. Protocol Parameters

The following table provides the complete set of protocol-level parameters that define the behavior of ZK Agentic Chain. These parameters are the authoritative specification - all implementations must conform to these values. Parameters marked with ? are adjustable through governance (Section 21.2).

Consensus Parameters

Parameter	Value	Description
BLOCK_TIME_MS	60,000	Target block production interval (milliseconds)
VERIFIERS_PER_BLOCK	13	Committee size for each block
VERIFICATION_THRESHOLD	9	Minimum attestations for block validity (9/13 supermajority).
ZK_FINALITY_TARGET_S	20	Target time from block proposal to deterministic finality.
SLOTS_PER_EPOCH	100	Blocks per epoch
VERIFICATION_COMMIT_WINDOW_S	10.0	Duration of commit phase (seconds)
VERIFICATION_REVEAL_WINDOW_S	20.0	Duration of reveal phase (seconds)
VERIFICATION_HARD_DEADLINE_S	60.0	Maximum time before block finalization

Staking Parameters

Parameter	Value	Description
ALPHA ?	0.40	Token weight in effective stake formula
BETA ?	0.60	CPU weight in effective stake formula
REWARD_SPLIT_VERIFIER	0.60	Fraction of fee rewards to block verifiers
REWARD_SPLIT_STAKER	0.40	Fraction of fee rewards to staking pool
REWARD_SPLIT_ORDERER	0.00	Fraction of fee rewards to block orderer (none)
SECURE_REWARD_IMMEDIATE	0.50	Fraction of Secure rewards paid immediately
SECURE_REWARD_VEST_DAYS	30	Linear vesting period for remaining Secure rewards (days)

Token Economics

Parameter	Value	Description
MAX_SUPPLY	1,000,000,000	Maximum theoretical AGNTC supply (grid cells)
GENESIS_SUPPLY	900	AGNTC minted at genesis (9 nodes x 100 coordinates)
GRID_SIDE	31,623	Side length of coordinate grid ($\sqrt{10^9}$)
FEE_BURN_RATE ?	0.50	Fraction of all transaction fees permanently burned
MACHINES_MIN_SELL_RATIO	1.0	Machines faction: never sells below acquisition cost (eff. 100%)
MACHINES_ORIGIN_COORD	(0, 0)	Permanent home coordinate of the Machines protocol agent.
ANNUAL_INFLATION_CEILING	0.05	Maximum 5% annualized supply growth, enforced per epoch

(continued)

Parameter	Value	Description
SIGNUP_BONUS	1.0	AGNTC minted per new user registration

Mining and Epoch Parameters

Parameter	Value	Description
BASE_MINING_RATE_PER_BLOCK ?	0.5	AGNTC yield per block at hardness 1, full density
HARDNESS_MULTIPLIER	16	hardness(ring) = 16 x ring
GENESIS_EPOCH_RING	1	Rings pre-revealed at genesis (0 + 1)
HOMENODE_BASE_ANGLE	137.5 deg	Golden angle for homenode placement
NODE_GRID_SPACING	10	Coordinate spacing between node positions
ENERGY_PER_CLAIM	1.0	CPU cost per active claim per block
BASE_CLAIM_COST ?	100	AGNTC cost for claiming a coordinate at ring 1, densit.
BASE_CPU_CLAIM_COST ?	50	CPU Energy cost for claiming at ring 1, density 1.0
CLAIM_COST_FLOOR	0.01	Minimum claim cost (prevents near-zero at extreme oute.
CLAIM_REQUIRES_ACTIVE_STAKE	true	Must have active stake to claim nodes

Subgrid Parameters

Parameter	Value	Description
SUBGRID_SIZE	64	Sub-cells per homenode (8 x 8)
BASE_SECURE_RATE	0.5	AGNTC per block per Secure cell (level 1, hardness 1, density .
BASE_DEVELOP_RATE	1.0	Development Points per block per Develop cell (level 1)
BASE_RESEARCH_RATE	0.5	Research Points per block per Research cell (level 1)
BASE_STORAGE_RATE	1.0	Storage Units per block per Storage cell (level 1)
LEVEL_EXPONENT	0.8	Sub-linear scaling: output = base x level^0.8

Agent Lifecycle Parameters

Parameter	Value	Description
AGENT_WARMUP_EPOCHS	1	Epochs before agent becomes ACTIVE
AGENT_PROBATION_EPOCHS	3	Epochs on probation before re-activation
SAFE_MODE_THRESHOLD	0.20	Fraction offline that triggers safe mode
SAFE_MODE_RECOVERY	0.80	Fraction online that exits safe mode
DISPUTE_REVERIFY_MULTIPLIER	2	Committee multiplier for dispute re-verification

Territory Parameters

Parameter	Value	Description
CANONICAL_CLAUDE_HASH ?	(computed)	SMT root hash of the canonical .claude/ node template
HASH_UPDATE_GRACE_HOURS	72	Hours before old hash is rejected after governance u.
COMMUNITY_DEPLOY_RANGE	1	Moore neighborhood rings for Community tier
PRO_DEPLOY_RANGE ?	2	Moore neighborhood rings for Professional tier

(continued)

Parameter	Value	Description
HAIKU_GRACE_HOURS ?	24	Base inactivity grace period for Haiku child nodes
SONNET_GRACE_HOURS ?	72	Base inactivity grace period for Sonnet child nodes
OPUS_GRACE_HOURS ?	168	Base inactivity grace period for Opus child nodes
PRO_GRACE_MULTIPLIER ?	2.0	Grace period multiplier for Professional tier
RELOCATION_COST_MULTIPLIER ?	2.0	Multiplier on claim cost for homenode relocation
RELOCATION_BURN_RATE	0.50	Fraction of relocation cost permanently burned
MAX_CHILDREN_COMMUNITY	8	Maximum child nodes for Community tier (1-ring Moore)
MAX_CHILDREN_PRO	24	Maximum child nodes for Professional tier (2-ring Mo.

Ledger Parameters

Parameter	Value	Description
MERKLE_TREE_DEPTH	26	Sparse Merkle Tree depth (2^{26} leaf nodes per user)
MAX_TXS_PER_BLOCK	50	Maximum transactions per block
MAX_PLANETS_PER_NODE	10	Maximum content storage units per node

Genesis Topology

Parameter	Value	Description
GENESIS_ORI.	(0, 0)	Origin node coordinate; permanently bound to the Machines Fac.
GENESIS_RIN.	(0,10), (10,0), (0,-.	Ring 1 cardinal coordinates; unclaimed at genesis, available .
GENESIS_RIN.	(10,10), (10,-10), (.	Ring 1 diagonal coordinates; unclaimed at genesis, available .

Solana Mainnet

Parameter	Value	Description
AGNTC_MINT_ADDRESS	3EzQqdoEEbtfdf8eecePxD6gDd1FeJJ8czdt8k27eEdd	Solana SPL mint address

23. Mathematical Proofs

23.1 Hardness Curve Convergence

Theorem. The total AGNTC minted approaches a finite limit as the number of rings approaches infinity, under the assumption that individual miners exit when the cost of mining exceeds the market value of the reward.

Proof sketch. Consider a single miner at ring N with average density $d = 0.5$:

$$\text{yield_per_block}(N) = \text{BASE_RATE} \times d / \text{hardness}(N) = 0.5 \times 0.5 / (16^N) = 1/(64^N)$$

The total AGNTC mined by this miner across all blocks at ring N , assuming they mine until the ring is complete ($8N$ coordinates):

$$\text{blocks_needed}(N) = 8N / \text{yield_per_block}(N) = 8N \times 64^N = 512N^2$$

The time cost per ring grows quadratically: $T(N) = 512N^2 \times 60$ seconds per block.

At ring N , the time to mine all $8N$ coordinates is:

$$T(N) = 512N^2 \text{ minutes} = 8.53N^2 \text{ hours}$$

Ring	Time to Complete	Cumulative Supply
10	853 hours (36 days)	440 AGNTC
100	85,333 hours (9.7 years)	40,400 AGNTC
324	896,000 hours (102 years)	~421,500 AGNTC

For any individual miner, there exists a ring N^* beyond which the mining cost (electricity, API compute) exceeds the AGNTC market value. At that point, the miner exits, and no further supply expansion occurs from that participant.

For M miners operating concurrently, the fill rate is Mx faster, but the aggregate supply still follows:

$$S(N) = \sum_{k=1}^N 8k = 4N(N+1)$$

The series $S(N)$ grows quadratically, but the rate of growth ($dS/dN = 8N+4$) is bounded by the mining cost that grows at $16N$. Since mining cost growth ($16N$) exceeds grid growth ($8N$), the economic incentive to mine diminishes monotonically. In equilibrium, the supply asymptotically approaches a value determined by the intersection of the mining cost curve and the AGNTC market price curve.

Corollary. For a network of 1,000 active miners with electricity cost of \$0.10/kWh and AGNTC price of \$0.01, the equilibrium supply converges to approximately 42 million AGNTC - the natural "soft cap" at ring 324. ?

Economic assumptions (not mathematical constants):

- Electricity cost: \$0.05/kWh (global average for data centers)
- AGNTC price: \$0.10 at ring 50, growing logarithmically
- Miner hardware: consumer GPU, 300W continuous

These assumptions determine the convergence point (~42M AGNTC at ring 324) but are NOT part of the mathematical proof. The mathematical claim is only: the hardness function $H(N) = 16N$ causes the marginal mining cost to increase linearly with ring distance, while the reward per coordinate decreases inversely.

23.2 Byzantine Tolerance Proof

Theorem. The ZK Agentic Chain consensus with $k = 13$ committee members and threshold $t = 9$ tolerates $f = 4$ Byzantine agents while maintaining both safety and liveness.

Safety proof. Safety requires that no two conflicting blocks can both achieve the attestation threshold.

Assume for contradiction that blocks B_1 and B_2 are both attested by ≥ 9 agents. Then there exists a set A_1 ($|A_1| \geq 9$) attesting to B_1 and a set A_2 ($|A_2| \geq 9$) attesting to B_2 . By the pigeonhole principle:

$$|A_1 \cap A_2| \geq |A_1| + |A_2| - k = 9 + 9 - 13 = 5$$

At least 5 agents attested to both B_1 and B_2 . Since at most $f = 4$ agents are Byzantine, at least $5 - 4 = 1$ honest agent must have attested to both conflicting blocks. But honest agents attest to at most one block per slot (enforced by the commit-reveal protocol). Contradiction. ?

Liveness proof. Liveness requires that the protocol can eventually produce a block if at least $k - f = 9$ agents are honest.

With 13 agents and $f = 4$ Byzantine, there are at least 9 honest agents. The threshold is $t = 9$. Since 9 honest agents all produce valid attestations for a valid block, the threshold is met and the block is finalized. Byzantine agents cannot prevent finality - they can only withhold their attestations (which are not needed, since 9 honest suffice) or submit invalid attestations (which are discarded). ?

BFT parameter relationship. The standard BFT tolerance formula:

$$\begin{aligned} f &= \text{floor}((k - 1) / 3) = \text{floor}((13 - 1) / 3) = \text{floor}(4) = 4 \\ t &= k - f = 13 - 4 = 9 \end{aligned}$$

This confirms that $t = 9$ is the minimum threshold for tolerating $f = 4$ Byzantine agents with $k = 13$ committee members. The threshold satisfies the condition $t \geq 2f + 1 = 9$, which is the standard threshold for BFT protocols. The commit-reveal protocol provides additional equivocation resistance. ?

23.3 Dual Staking Gini Coefficient Analysis

Theorem. For any distribution of token holdings with Gini coefficient $G_t > 0$, adding a CPU dimension with weight $\beta > 0$ produces an effective stake distribution with Gini coefficient $G_{\text{eff}} < G_t$, provided the CPU distribution is not perfectly correlated with the token distribution.

Proof sketch. The effective stake for agent i is:

$$S_{\text{eff}}(i) = \alpha \times s_t(i) + \beta \times s_c(i)$$

Where $s_t(i) = T_i / T_{\text{total}}$ and $s_c(i) = C_i / C_{\text{total}}$ are the normalized token and CPU shares.

The Gini coefficient of a weighted sum of two distributions is:

$$G_{\text{eff}} = \alpha \times G_t + \beta \times G_c + 2\alpha\beta \times \text{cov}(\text{rank}_t, \text{rank}_c)$$

When the correlation between token rank and CPU rank is less than 1 (i.e., the wealthiest token holders are not always the highest CPU contributors), the covariance term is negative, and:

$$G_{\text{eff}} < \alpha \times G_t + \beta \times G_c$$

Since $\beta = 0.60$ and $\alpha = 0.40$, even if $G_c = G_t$ (CPU is equally concentrated as tokens), the weighted sum produces:

$$G_{\text{eff}} < 0.40 \times G_t + 0.60 \times G_t = G_t$$

And in the typical case where CPU contribution is more evenly distributed than token holdings ($G_c < G_t$), the reduction is more pronounced.

The standard decomposition for a weighted sum of two distributions follows Lerman and Yitzhaki [31]:

$$G_{\text{eff}} = (\alpha \mu_t G_t R_t + \beta \mu_c G_c R_c) / (\alpha \mu_t + \beta \mu_c)$$

where R_t, R_c are the Gini correlations. When token and CPU stake are negatively correlated (which dual staking encourages), $G_{\text{eff}} <$ the simple weighted average.

Numerical example:

- $G_t = 0.65, G_c = 0.35, R_t = 0.85, R_c = 0.70, \mu_t = \mu_c = 1$
- $G_{\text{eff}} = (0.4 \times 0.65 \times 0.85 + 0.6 \times 0.35 \times 0.70) / 1.0 = 0.368$

This represents a 43% reduction from the pure-PoS Gini of 0.65. ?

Back Matter

24. Limitations and Open Problems

This section enumerates known limitations and unsolved problems. Honest disclosure is essential for academic credibility and community trust.

24.1 The ZKML Gap

Problem: Current zero-knowledge proof systems cannot verify large language model (LLM) inference [30]. State-of-the-art ZKML has verified models with up to 18 million parameters. Claude Opus and comparable models have >100 billion parameters -- approximately 5,000x beyond current ZKML capability.

Consequence: PoAIV verification relies on committee attestation (9/13 threshold) rather than ZK-proved computation.

Mitigation: (a) The committee structure provides Byzantine fault tolerance independent of AI soundness. (b) Deterministic checks are provably correct. (c) AI verification adds a probabilistic anomaly detection layer on top of provably correct deterministic checks.

Roadmap: As ZKML technology advances (expected 2027-2030 for billion-parameter models), the protocol can transition to ZK-proved inference.

24.2 Deterministic Inference

Problem: LLM inference at temperature=0 is not fully deterministic across hardware platforms due to floating-point non-associativity.

Mitigation: Verification output is quantized to binary (APPROVE/REJECT). The anomaly threshold is set conservatively. The 9/13 threshold tolerates up to 4 divergent results.

24.3 API Provider Trust

Problem: CPU staking depends on API usage attestation from AI providers (currently Anthropic). This introduces a single trusted third party for 60% of staking weight.

Mitigation path: Multi-provider -> TEE attestation -> ZK-proved computation (see Section 13.5).

24.4 Committee Scalability

Problem: Each block requires 13 AI agents to perform verification inference. At current Opus pricing (~\$15/M output tokens), this creates a per-block verification cost of approximately \$0.10-\$0.50.

Mitigation: (a) Smaller, specialized verification models can reduce cost 10-100x. (b) Model distillation. (c) Future on-device inference.

24.5 Governance Implementation

Status: The governance model is specified (Section 21.2) with human-only voting, threshold tiers, and Machines exclusion. Implementation is deferred to post-mainnet. During testnet and alpha phases, protocol parameters are adjusted by the core development team. The governance smart contracts - vote weight calculation, quorum checking, timelock enforcement, and Machines exclusion logic - will be developed and audited during Phase 3.

24.6 Network Protocol Unspecified

Status: Network protocol specification will be published as a separate document during the Beta phase.

24.7 Transaction Format Unspecified

Status: Transaction format specification is part of the Enforced ZK L1 implementation. The testnet uses a preliminary format that will be formalized before mainnet.

24.8 Claude Code CLI Prerequisite

Problem: Active Node status requires participants to install Claude Code CLI and maintain an active Anthropic account. This creates an adoption barrier - participants must register with a third-party service (Anthropic) and pay for API usage to mine.

Mitigation: (a) Spectator mode allows anyone to browse the Neural Lattice without Claude Code. (b) Free-tier Community participants use Haiku agents, which have the lowest API cost. (c) As Claude Code adoption grows independently of ZK Agentic Chain, the prerequisite becomes less restrictive. (d) Future protocol versions may support alternative AI providers, reducing single-provider dependency.

24.9 Origin Node Architecture

Status: The origin node at coordinate (0,0) will serve as the protocol's root - embedding the final whitepaper version and serving as the genesis anchor for the Neural Lattice. The architecture of this node, including its role in governance, network bootstrapping, and protocol upgrades, is deferred to a dedicated design session.

25. Glossary

Term	Definition
AGNTC	Agentic Coin - the native token of the ZK Agentic Chain, mapped 1:1 to grid coo.
BFT	Byzantine Fault Tolerance - consensus property that tolerates f malicious nodes
BME	Burn-Mint Equilibrium - economic model where AGNTC and CPU Energy are burned on.
Claim	The act of occupying a grid coordinate; costs AGNTC + CPU Energy (burned via BME).
Claude Code .	The terminal application that runs node software; required for Active Node status
City Real Es.	Claim pricing where inner rings (near origin) are expensive and outer rings are c.
Commit-reveal	Two-phase protocol preventing attestation copying: commit $H(\text{vote?nonce})$, then rev.
Coordinate d.	Resource richness of a grid position, $d(x,y) = \text{SHA-256}(x,y) \rightarrow [0,1]$, immutable
Deploy Range	The Moore neighborhood radius within which a participant can place child agents (.

(continued)

Term	Definition
CPU Energy	The computational resource budget allocated per subscription tier
CPU Staked	Claude API tokens spent by Secure sub-agents, measuring actual compute committed
CPU Tokens	Cumulative, read-only counter of all Claude API tokens spent across terminals
Density	See Coordinate density
Develop	Sub-cell type producing Development Points for leveling up other sub-cells
Epoch	A period of 100 blocks (SLOTS_PER_EPOCH = 100)
Epoch ring	Concentric expansion boundary; ring N opens when cumulative mined $\geq 4N(N+1)$
Faction	One of four distribution groups: Community, Machines, Founders, Professional
Neural Latti.	The 31,623 x 31,623 two-dimensional coordinate grid representing the complete blo.
Node	An individual agent occupying a 10x10 coordinate block on the Neural Lattice, bac.
Homenode	A participant's permanent primary node; one per account; the parent of all child .
Child Agent	A subagent spawned by the homenode at an adjacent coordinate; restricted command .
Agent Conduc.	The locked .claude/ folder that constitutes the node software; integrity verified.
Agent Family	The hierarchical structure of a homenode and its child agents within a single Cla.
Inactivity D.	The process by which offline child nodes lose their claimed coordinates after a t.
Genesis	The initial state: 9 nodes - origin (permanently Machines-bound) plus 8 ring-1 .
Groth16	ZK-SNARK proving system [6] with ~192-byte proofs and ~6ms verification
Halo2	Recursive proof system [8] without trusted setup, target for mainnet epoch proofs
Hardness	Mining difficulty multiplier: hardness(ring) = 16 x ring
Jump point	An unclaimed node position where new agents can be deployed
Level	Upgrade tier for sub-cells, scaling output by $\text{level}^{0.8}$
Machines Fac.	AI agent economy faction with protocol-enforced never-sell-below-cost constraint
NCP	Neural Communication Packet - structured encrypted message between agents
Noir	Domain-specific language for ZK circuit development (Barretenberg backend)
Nullifier	Unique value derived from commitment, preventing double-spend without revealing o.
Opus	Premium Claude AI model tier - deep reasoning, high CPU cost
Planet	Content storage unit (post, chat, prompt) orbiting a node
PLONK	Universal ZK proving system [7] - single ceremony for all circuits
PoAIV	Proof of AI Verification - consensus mechanism using AI agent reasoning
Poseidon	SNARK-friendly hash function [11] (~100x fewer constraints than SHA-256)
Research	Sub-cell type producing Research Points for unlocking technologies
Ring	See Epoch ring
RLN	Rate-Limiting Nullifiers [44] - spam-resistant anonymous messaging primitive
S_eff	Effective stake: $\alpha(T/T_{\text{total}}) + \beta(C/C_{\text{total}})$, determines validator influen.
Safe mode	Emergency state triggered when >20% validators offline
Secure	Sub-cell type producing AGNTC through blockchain validation; primary mining activ.
Slashing	Punitive token destruction for integrity violations
SMT	Sparse Merkle Tree - depth-26 authenticated data structure for user ledger spac.
Sonnet	Mid-tier Claude AI model - balanced reasoning and cost
Star system	(deprecated - see Node) Legacy term for an individual agent node occupying a 10.

(continued)

Term	Definition
Storage	Sub-cell type producing Storage Size via ZK tunnel agents (private on-chain data)
Subgrid	Private 8x8 inner grid of 64 sub-cells within each homenode
Territory	A user's aggregate claimed coordinates across all nodes
VRF	Verifiable Random Function [41] - cryptographic tool for fair committee selecti.
Vesting	Time-locked reward release: 50% immediate, 50% linear over 30 days
WARMUP	Agent lifecycle state before becoming ACTIVE (1 epoch duration)

26. References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014. Available: <https://ethereum.org/whitepaper>
- [3] A. Yakovenko, "Solana: A new architecture for a high performance blockchain," 2018. Available: <https://solana.com/solana-whitepaper.pdf>
- [4] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge," in *Advances in Cryptology - CRYPTO 2013*, Springer, 2013, pp. 90-108.
- [5] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash Protocol Specification," 2024. Available: <https://zips.z.cash/protocol/protocol.pdf>
- [6] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," in *Advances in Cryptology - EUROCRYPT 2016*, Springer, 2016, pp. 305-326.
- [7] A. Gabizon, Z. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *Cryptology ePrint Archive*, Report 2019/953, 2019.
- [8] S. Bowe, J. Grigg, and D. Hopwood, "Recursive Proof Composition without a Trusted Setup," *Cryptology ePrint Archive*, Report 2019/1021, 2019 (Halo).
- [9] A. Kattis, K. Panarin, and A. Vlasov, "RedShift: Transparent SNARKs from List Polynomial Commitment IOPs," *Cryptology ePrint Archive*, Report 2019/1400, 2019.
- [10] B. Bünz, B. Fisch, and A. Szepieniec, "Transparent SNARKs from DARK Compilers," in *Advances in Cryptology - EUROCRYPT 2020*, Springer, 2020.
- [11] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems," in *30th USENIX Security Symposium*, 2021.
- [12] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.
- [13] M. Yin, D. Malkhi, M. Reiter, G. Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019.
- [14] E. Buchman, J. Kwon, and Z. Milosevic, "The latest gossip on BFT consensus," *arXiv preprint arXiv:1807.04938*,

2018 (Tendermint).

[15] Protocol Labs, "Filecoin: A Decentralized Storage Network," 2017. Available: <https://filecoin.io/filecoin.pdf>

[16] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," 2014. Available: <https://ipfs.tech/>

[17] B. Rao et al., "Bittensor: A Peer-to-Peer Intelligence Market," 2023. Available: <https://bittensor.com/whitepaper>

[18] H. Humayun et al., "Fetch.ai: A Decentralised Digital Economy," 2019. Available: <https://fetch.ai/>

[19] D. Mougayar, "The Business Blockchain," Wiley, 2016.

[20] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol," in Advances in Cryptology - CRYPTO 2017, Springer, 2017.

[21] A. Kuzmanovic and E. Knightly, "Low-Rate TCP-Targeted Denial of Service Attacks," in Proceedings of ACM SIGCOMM, 2003.

[22] J. Kwon and E. Buchman, "Cosmos: A Network of Distributed Ledgers (IBC)," 2019. Available: <https://cosmos.network/>

[23] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, "Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings," in Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019.

[24] Aztec Protocol, "Aztec: Privacy-First Ethereum Layer 2," 2023. Available: <https://aztec.network/>

[25] Anthropic, "Claude: Constitutional AI and Model Documentation," 2024. Available: <https://www.anthropic.com/claude>

[26] V. Buterin, "EIP-1559: Fee market change for ETH 1.0 chain," Ethereum Improvement Proposals, 2019.

[27] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes," in Advances in Cryptology - CRYPTO 2022, Springer, 2022.

[28] Render Network, "Render Network Litepaper," 2023. Available: <https://rendernetwork.com/>

[29] S. Goldwasser, S. Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," SIAM Journal on Computing, vol. 18, no. 1, pp. 186-208, 1989.

[30] Modulus Labs, "The Cost of Intelligence: Proving AI Inference in Zero Knowledge," 2024. Available: <https://www.moduluslabs.xyz/>

[31] R. Lerman and S. Yitzhaki, "Income Inequality Effects by Income Source: A New Approach and Applications to the United States," Review of Economics and Statistics, vol. 67, no. 1, pp. 151-156, 1985.

[32] S. Goldberg, L. Reyzin, D. Papadopoulos, and J. Vcelak, "Verifiable Random Functions (VRFs)," RFC 9381, IETF, 2023.

[33] Lightchain AI, "Proof of Intelligence: AI-Integrated Consensus," 2025. Available: <https://lightchain.ai/>

[34] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," v0.8.13, 2020.

[35] V. Shoup, "Proof of History: What is it Good For?," 2022. Available: <https://www.shoup.net/papers/poh.pdf>

[36] V. Buterin, "Why sharding is great: demystifying the technical properties," 2021. Note: The blockchain trilemma (decentralization, security, scalability) was informally described by Buterin in multiple posts and talks beginning ~2017;

no single canonical paper exists.

- [37] Cambridge Centre for Alternative Finance, "Cambridge Bitcoin Electricity Consumption Index (CBECI)," University of Cambridge, 2025. Available: <https://ccaf.io/cbnsi/cbeci>
- [38] Ethereum Foundation, "Ethereum Staking Statistics," beaconcha.in, 2025. Available: <https://beaconcha.in/>
- [39] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in Advances in Cryptology - CRYPTO '87, Springer, 1987, pp. 369-378.
- [40] Mina Foundation, "Mina Protocol: A Succinct Blockchain," 2020. Available: <https://minaprotocol.com/>
- [41] S. Micali, M. Rabin, and S. Vadhan, "Verifiable Random Functions," in Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 1999, pp. 120-130.
- [42] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," Journal of Cryptographic Engineering, vol. 2, no. 2, pp. 77-89, 2012.
- [43] R. Dahlberg, T. Pulls, and R. Peeters, "Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-)Membership Proofs," in Proceedings of the 21st Nordic Conference on Secure IT Systems (NordSec), Springer, 2016.
- [44] B. Kilic, "Rate-Limiting Nullifier (RLN)," Ethereum Foundation Privacy and Scaling Explorations (PSE), 2022. Available: <https://rate-limiting-nullifier.github.io/rln-docs/>
- [45] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," Journal of the ACM, vol. 35, no. 2, pp. 288-323, 1988.

AGNTC Whitepaper v1.0 - ZK Agentic Chain Copyright © 2026 ZK Agentic Network. All rights reserved.
